

Team Notebook

SUST_LavNai

May 14, 2026

Contents

1 DP	2	3 Geo	9	6.4 extendEuclid	18
1.1 SOSDP	2	3.1 Closest Pair	9	6.5 phi	18
2 Data Structure	2	3.2 Geometry 2D	9	6.6 segmented sieve	18
2.1 2D BIT	2	4 Graph Trees	14	7 String	18
2.2 ArticulationPoint	2	4.1 Cycle Detection	14	7.1 2D Hashing	18
2.3 BIT	2	4.2 DJ xtra	14	7.2 Aho Corasick	19
2.4 Custom Hash for Maps	2	4.3 LCA	14	7.3 Bit Trie	19
2.5 DSU	3	4.4 MaxFlow	14	7.4 KMP	20
2.6 HLD	3	4.5 Min Cost Flow	15	7.5 Longest Palindromes	20
2.7 Lazy SegTree	3	4.6 Tree Diametre	15	7.6 Min Lexo Rotation	20
2.8 MergeSortTree	4	5 Math	16	7.7 String Hashing	20
2.9 PrefixSum3D	4	5.1 FFT	16	7.8 Trie	20
2.10 Range BIT	4	5.2 MMGeometricSum	16	7.9 Z Func	21
2.11 Segtree Hashing	5	5.3 Mat Xpo	16	7.10 manacharAlgo	21
2.12 Sparse Table	5	5.4 NTT	16	7.11 multiSetHash	21
2.13 TarjansAlgo	5	5.5 Nth Fibonacci	17	7.12 suffixArray	22
2.14 centroidDecomposition	5	5.6 Rand	17	7.13 suffix _a rray2	23
2.15 kosarajus	6	5.7 XorBasis	17	8 ZMiscellenous	23
2.16 mosAlgo	7	6 Number Theory	17	8.1 build	23
2.17 palindromicTree	7	6.1 Factorial Mod	17	8.2 formulas	23
2.18 persistentSegTree	7	6.2 Stirling2nd	17		
2.19 virtualTree	8	6.3 bigInt	17		

1 DP

1.1 SOSDP

```
for(int i = 1; i < (1 << sz); i++){
    for(int j = 1; j <= sz; j++){
        if((i >> (j - 1)) & 1){
            dpOr[i][j] = dpOr[i][j - 1] + dpOr[i ^ (1 << (j - 1))][j - 1]; // ekhane j - 1 diye j - 1th
            bit sorache abar ager value theke ana hoche,
            ekhane j, 1 based ei jonno value soranor time
            a j - 1 kora hoise
        }
        else{dpOr[i][j] = dpOr[i][j - 1];}
    }
}
for(int i = (1 << sz) - 1; i > 0; i--){ // ekhane loop ulta
    dik diye cholse karon [i ^ (1 << (j - 1))] eta boro
    value te jeye value niye ashe, tai boro gulo age
    calculate korte hobe
    for(int j = 1; j <= sz; j++){
        if((i >> (j - 1)) & 1){dpAnd[i][j] = dpAnd[i][j - 1];}
        else{
            dpAnd[i][j] = dpAnd[i][j - 1] + dpAnd[i ^ (1 << (j - 1))][j - 1];
        }
    }
}
```

2 Data Structure

2.1 2D BIT

```
template<typename T> // BIT2D<int32_t> bt(N,M);
struct BIT2D{ // 1-base
    int n,m;
    vector<vector<T>> t;
    BIT2D(int n,int m):n(n),m(m),t(n+2,vector<T>(m+2,0)){}
    void update(int x,int y,T val){
        for(int i=x;i<=n;i+=(i&-i))for(int j=y;j<=m;j+=(j&-j))t[i][j]+=val;
    }
    void range_update(int x1,int y1,int x2,int y2,T val){
        update(x1,y1,val); update(x1,y2+1,-val);
        update(x2+1,y1,-val); update(x2+1,y2+1,val);
    }
}
```

```
T query(int x,int y)const{
    T sum=0;
    for(int i=x;i>0;i--(i&-i))for(int j=y;j>0;j--(j&-j))sum
        +=t[i][j];
    return sum;
}
T query(int x1,int y1,int x2,int y2)const{
    return query(x2,y2)-query(x1-1,y2)-query(x2,y1-1)+query
        (x1-1,y1-1);
}
void reset(){
    for(int i=0;i<=n;++i)fill(t[i].begin(),t[i].end(),0);
}
};
```

2.2 ArticulationPoint

```
vector<int> tin, low, vis, mark;
vector<pair<int, int >> bridge;
int timer = 1;

void dfs(int node, int parentNode, vector<int> adj[]){
    tin[node] = low[node] = timer++;
    vis[node] = 1;
    int child = 0;
    for(auto it: adj[node]){
        if(it == parentNode)continue;
        if(!vis[it]){
            dfs(it, node, adj);
            low[node] = min(low[node], low[it]);
            if (low[it] >= tin[node] && parentNode != -1) {
                mark[node] = 1;
            }
            child++;
        }
        else{
            low[node] = min(low[node], tin[it]);
        }
    }
    if (child > 1 && parentNode == -1) {
        mark[node] = 1;
    }
    return;
}

low = vector<int>(n + 1, 0);
tin = vector<int>(n + 1, 0);
vis = vector<int>(n + 1, 0);
mark = vector<int>(n + 1, 0);
```

```
vector<int> adj[n + 1];
```

```
bridge.clear();
timer = 1;
```

2.3 BIT

```
struct BIT{ // 1-base query
    int sz; vector<int> bit;
    BIT(int n){ sz=n; bit.resize(sz+1,0); }
    void update(int pos, int v){
        while(pos<=sz){
            bit[pos]+=v;
            pos=pos+(pos&(-pos));
        }
    }
    int sum(int pos){ int s=0;
        while(pos>0){
            s+=bit[pos];
            pos=pos-(pos&(-pos));
        }
        return s;
    }
    int query(int l, int r) return sum(r)-sum(l-1);
};
```

2.4 Custom Hash for Maps

```
struct custom_hash{
    static uint64_t sm64(uint64_t x){x+=0x9e3779b97f4a7c15; x=(x
        ^>>30)*0xbfb58476d1ce4e5b9; x=(x^>>27)*0
        x94d049bb133111eb; return x^>>31);}
    size_t operator()(uint64_t x) const{static const uint64_t R=
        chrono::steady_clock::now().time_since_epoch().count();
        return sm64(x+R);}
    template<class T1,class T2> size_t operator()(const pair<T1,
        T2>&p) const{size_t h1=(*this)((uint64_t)p.first),h2=(*
        this)((uint64_t)p.second); return h1^(h2+0x9e3779b9+(h1
        <<6)+(h1>>2));}
    template<class...Ts> size_t operator()(const tuple<Ts...>&t)
        const{size_t s=0; apply([&](const auto&...e){((s^=(*
        this)((uint64_t)e+0x9e3779b9+(s<<6)+(s>>2)),...);},t);
        return s;}
    template<class T> size_t operator()(const vector<T>&v) const
        {size_t s=v.size(); for(auto&x:v) s^=(*this)((uint64_t)
        x+0x9e3779b9+(s<<6)+(s>>2); return s;}
};
```

2.5 DSU

```
struct DSU {
    vector<int> par,sz; DSU(){}
    DSU(int n){ init(n); }
    void init(int n){
        par.resize(n);
        iota(par.begin(),par.end(),0);
        sz.assign(n,1);
    }
    int find(int v){
        while(v!=par[v]){
            v=par[v]=par[par[v]];
        }
        return v;
    }
    bool merge(int a,int b){
        a=find(a); b=find(b);
        if(a==b) return false;
        if(sz[a]<sz[b]) swap(a, b);
        par[b]=a; sz[a]+=sz[b];
        return true;
    }
};
```

2.6 HLD

```
template<typename T>
struct segTree{
    int n; vector<T> t, lazy;
    T neutral =0; // 0 for sum, -inf for max, +inf for min
    SegTree(int n =0): n(n), t(4*n, neutral), lazy(4*n, 0){}
    T combine(T a, T b){}
    void push(int v, int tl, int tr){}
    void build(const vector<T>& a, int v, int tl, int tr){}
    void build(const vector<T>& a) { build(a, 1, 1, n); }
    void update(int v, int tl, int tr, int l, int r, T add){}
    void update(int l, int r, T add){update(1, 1, n, l, r, add);}
    T query(int v, int tl, int tr, int l, int r){}
    T query(int l, int r) { return query(1, 1, n, l, r); }
    void set_value(int pos, T val) { // overwrite
        T cur =query(pos, pos);
        update(pos, pos, val - cur);
    }
};
template<typename T>
struct HLD {
    int n,timer; vector<vector<int>>> g;
```

```
vector<int> parent, depth, heavy, head, pos, sz;
vector<T> flat; SegTree<T> seg;
HLD(int n): n(n), g(n+1), parent(n+1), depth(n+1),heavy(n
+1,-1), head(n+1), pos(n+1), sz(n+1), timer(0) {}
void add_edge(int u, int v) {
    g[u].push_back(v);
    g[v].push_back(u);
}
int dfs(int u, int p) {
    parent[u] =p; sz[u] =1;
    int max_sz =0;
    for (int v : g[u]) if (v !=p) {
        depth[v] =depth[u] + 1;
        int sub =dfs(v, u); sz[u] +=sub;
        if (sub > max_sz) max_sz =sub, heavy[u] =v;
    }
    return sz[u];
}
void decompose(int u, int h, const vector<T>& val) {
    head[u] =h; pos[u] ++=timer; flat[timer] =val[u];
    if (heavy[u] !=-1) decompose(heavy[u], h, val);
    for (int v : g[u])if (v !=parent[u] && v !=heavy[u])
        decompose(v, v, val);
}
void init(const vector<T>& val, int root =1) {
    flat.assign(n+1, 0);
    dfs(root, 0);
    timer =0;
    decompose(root, root, val);
    seg =SegTree<T>(n);
    seg.build(flat);
}
void update_path(int u, int v, T add) { // Path range add
    while (head[u] !=head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        seg.update(pos[head[u]], pos[u], add);
        u =parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
    seg.update(pos[u], pos[v], add);
}
T query_path(int u, int v) { // Path query (need to
update)
    T res =0; // check
    while (head[u] !=head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        res +=seg.query(pos[head[u]], pos[u]);
        u =parent[head[u]];
    }
    if (depth[u] > depth[v]) swap(u, v);
```

```
res +=seg.query(pos[u], pos[v]);
return res;
}
void update_subtree(int u, T add) { // Subtree range add
    seg.update(pos[u], pos[u] + sz[u] - 1, add);
}
T query_subtree(int u) { // Subtree query
    return seg.query(pos[u], pos[u] + sz[u] - 1);
}
void set_value(int u, T val) { // Point set
    seg.set_value(pos[u], val);
}
int lca(int u, int v) {
    while (head[u] !=head[v]) {
        if (depth[head[u]] < depth[head[v]]) swap(u, v);
        u =parent[head[u]];
    }
    return depth[u] < depth[v] ? u : v;
}
};
```

2.7 Lazy SegTree

```
class segTree{
public:
    int sz;
    vector<int> t,lazy;
    segTree(){}
    segTree(int l){
        sz=l;
        if(__builtin_popcount(sz)!=1){
            int x=31-__builtin_clz(sz);
            sz=(1LL<<(x+1));
        }
        t.assign(2*sz+1,0);
        lazy.assign(2*sz+1,0);
    }
    void apply(int root, int tl, int tr){
        if(!lazy[root]) return;
        t[root]+=lazy[root];
        if(tl!=tr){
            lazy[root*2]+=lazy[root];
            lazy[root*2+1]+=lazy[root];
        }
        lazy[root]=0;
    }
    int combine(int& left, int& right){
        int tm=left+right; return tm;
    }
};
```

```

void update(int root, int tl, int tr, int l, int r, int x)
{
    apply(root,tl,tr);
    if(l>tr || r<tl)    return;
    if(l<=tl && r>=tr){
        lazy[root]+=x;
        apply(root,tl,tr);
        return;
    }
    int mid=(tl+tr)/2;
    update(root*2,tl,mid,l,r,x);
    update(root*2+1,mid+1,tr,l,r,x);
    t[root]=combine(t[root*2],t[root*2+1]);
}

int query(int root, int tl, int tr, int l, int r){
    apply(root,tl,tr);
    if(l>tr || r<tl)    return 0;
    if(l<=tl && r>=tr)    return t[root];
    int mid=(tl+tr)/2;
    auto q1=query(root*2,tl,mid,l,r);
    auto q2=query(root*2+1,mid+1,tr,l,r);
    return combine(q1,q2);
}

void update(int l, int r, int x){
    update(1,0,sz-1,l,r,x);
}

int query(int l, int r){
    return query(1,0,sz-1,l,r);
}
};

```

2.8 MergeSortTree

```

const int N=1e5+5;
int n,q,a[N];
vector<int> seg[4*N+5];
//build merge sort tree
void build(int node,int l,int r){
    //if range length is 1 there's only one element to add
    and no children
    if (l==r){
        seg[node].push_back(a[l]);
        return;
    }int mid=(l+r)/2;
    build(node*2,l,mid);
    build(node*2+1,mid+1,r);
    int i=0,j=0;
    // use two pointers to merge the two vectors in O(r-l+1)
    while (i<seg[node*2].size() && j<seg[node*2+1].size()){

```

```

        if (seg[node*2][i]<seg[node*2+1][j]) seg[node].
            push_back(seg[node*2][i++]);
        else seg[node].push_back(seg[node*2+1][j++]);
    }
    while (i<seg[node*2].size()) seg[node].push_back(seg[node
        *2][i++]);
    while (j<seg[node*2+1].size()) seg[node].push_back(seg[
        node*2+1][j++]);
    return;
}

//query
int query(int node,int l,int r,int lx,int rx,int x){
    //if outside -> 0
    if (l>rx || r<lx)    return 0;
    //if inside do binary search
    if (l>=lx && r<=rx){
        int L=0,R=seg[node].size()-1,mid,ans=0;
        while (L<=R){
            mid=(L+R)/2;
            if (seg[node][mid]>x){
                ans = seg[node].size() - mid;
                R = mid - 1;
            }else L = mid + 1;
        }
        return ans;
    }
    int mid=(l+r)/2;
    return query(node*2,l,mid,lx,rx,x)+query(node*2+1,mid+1,r
        ,lx,rx,x);
}

```

2.9 PrefixSum3D

```

void solve()
{
    ll n;cin>>n;
    ll a[n + 1][n + 1][n + 1], pre[n + 1][n + 1][n + 1];
    for(ll i = 0; i <= n; i++){
        for(ll j = 0; j <= n; j++){
            for(ll k = 0; k <= n; k++){
                if(i == 0 || j == 0 || k == 0){pre[i][j][k] = 0;continue
                    ;}
                cin>>a[i][j][k];
                pre[i][j][k] = pre[i - 1][j][k] + pre[i][j - 1][k] + pre[
                    i][j][k - 1]
                    - pre[i - 1][j - 1][k] - pre[i - 1][j][k - 1] - pre[i
                        ][j - 1][k - 1]
                    + pre[i - 1][j - 1][k - 1] + a[i][j][k];
            }
        }
    }
}

```

```

    }
}
ll q;cin>>q;
while(q--){
    ll l1,r1,l2,r2,l3,r3;cin>>l1>>r1>>l2>>r2>>l3>>r3;
    ll ans = pre[r1][r2][r3] - pre[l1 - 1][r2][r3] - pre[r1][
        l2 - 1][r3] - pre[r1][r2][l3 - 1]
        + pre[l1 - 1][l2 - 1][r3] + pre[l1 - 1][r2][l3 - 1] +
        pre[r1][l2 - 1][l3 - 1]
        - pre[l1 - 1][l2 - 1][l3 - 1];
    cout<<ans<<nl;
}
}
}

```

2.10 Range BIT

```

const int N = 3e5 + 9; // 1-base query
struct RangeBIT {
    int64_t M[N], A[N];
    RangeBIT() {
        memset(M, 0, sizeof(M));
        memset(A, 0, sizeof(A));
    }
    void update(int i, int64_t mul, int64_t add) {
        for (; i < N; i +=i & (-i)) {
            M[i] +=mul;
            A[i] +=add;
        }
    }
    void upd(int l, int r, int64_t x) {
        update(l, x, -x * (l - 1));
        update(r + 1, -x, x * r);
    }
    int64_t query(int i) {
        int64_t mul =0, add =0;
        for (; i > 0; i -=i & (-i)) {
            mul +=M[i];
            add +=A[i];
        }
        return (mul * i + add);
    }
    int64_t query(int l, int r) {
        return query(r) - query(l - 1);
    }
};

```

2.11 Segtree Hashing

```
const int SZ=int(1e6)+9;
const int B1=151,mod1=127657753;
const int B2=239,mod2=987654319;
vector<int> P1(SZ,1),P2(SZ,1);
void pow_cal(){
    for(int i=1;i<SZ;i++){
        P1[i]=(P1[i-1]*B1)%mod1;
        P2[i]=(P2[i-1]*B2)%mod2;
    }
}
class segHash{
public:
    int n;
    vector<int> T1,T2;
    segHash(const string& s){
        n=s.size();
        T1.assign(n*4+1,0);
        T2.assign(n*4+1,0);
        build(s,1,0,n-1);
    }

    void build(const string& s, int v, int tl, int tr){
        if(tl==tr){
            T1[v]=(s[tl]-'a'+1)%mod1;
            T2[v]=(s[tl]-'a'+1)%mod2;
            return;
        }
        int mid=(tl+tr)/2;
        build(s,v*2,tl,mid);
        build(s,v*2+1,mid+1,tr);
        T1[v]=(((T1[v*2]*P1[tr-mid])%mod1)+T1[v*2+1])%mod1;
        T2[v]=(((T2[v*2]*P2[tr-mid])%mod2)+T2[v*2+1])%mod2;
    }

    void update(int v, int tl, int tr, int pos, char c){
        if(tl==tr){
            T1[v]=(c-'a'+1)%mod1;
            T2[v]=(c-'a'+1)%mod2;
            return;
        }
        int mid=(tl+tr)/2;
        if(pos<=mid){
            update(v*2,tl,mid,pos,c);
        }else{
            update(v*2+1,mid+1,tr,pos,c);
        }
        T1[v]=(((T1[v*2]*P1[tr-mid])%mod1)+T1[v*2+1])%mod1;
        T2[v]=(((T2[v*2]*P2[tr-mid])%mod2)+T2[v*2+1])%mod2;
    }
};
```

```
}

pair<int,int> query(int v, int tl, int tr, int l, int r){
    if(tl>r || tr<l){
        return {0,0};
    }
    if(l<=tl && tr<=r){
        return {T1[v],T2[v]};
    }
    int mid=(tl+tr)/2;
    auto left=query(v*2,tl,mid,l,min(mid,r));
    auto right=query(v*2+1,mid+1,tr,max(l,mid+1),r);
    int len=max(int(0),r-max(l,mid+1)+1);
    int H1=(((left.first*P1[len])%mod1)+right.first)%mod1;
    int H2=(((left.second*P2[len])%mod2)+right.second)%mod2;
    ;
    return {H1,H2};
}

void update(int pos, char c){
    return update(1,0,n-1,pos-1,c);
}

pair<int,int> geth(int l,int r){
    return query(1,0,n-1,l-1,r-1);
}
};
```

2.12 Sparse Table

```
const int N =1e5 + 9;
int t[N][18], a[N];
void build(int n) { //RMQ
    for(int i =1; i <=n; ++i) t[i][0] =a[i];
    for(int k =1; k < 18; ++k) {
        for(int i =1; i + (1 << k) - 1 <=n; ++i) {
            t[i][k] =min(t[i][k - 1], t[i + (1 << (k - 1))][k - 1]);
        }
    }
}

int query(int l, int r) {
    int k =31 - __builtin_clz(r - l + 1);
    return min(t[l][k], t[r - (1 << k) + 1][k]);
}
```

2.13 Tarjans Algo

```
vector<int> tin, low, vis;
```

```
vector<pair<int, int > > bridge;
int timer = 1;
void findBridge(int node, int parentNode, vector<int> adj[])
{
    tin[node] = low[node] = timer++;
    vis[node] = 1;
    for(auto it: adj[node]){
        if(it == parentNode)continue;
        if(!vis[it]){
            findBridge(it, node, adj);
        }
        low[node] = min(low[node], low[it]);
        if(tin[node] < low[it])bridge.push_back({node, it});
    }
}

low = vector<int>(n + 1, 0);
tin = vector<int>(n + 1, 0);
vis = vector<int>(n + 1, 0);
vector<int> adj[n + 1];
bridge.clear();
timer = 1;
```

2.14 centroidDecomposition

```
vector<vector<int>> adj, up;
vector<int> tin, tout, sts, dep;
void dfs(int cur, int par = 1)
{
    tin[cur] = ++timer;
    // for node == 1 take par also equals to 1
    up[cur][0] = par;
    dep[cur] = dep[par] + 1;
    for (int i = 1; i <= 25; ++i)
        up[cur][i] = up[up[cur][i - 1]][i - 1];
    for (int child : adj[cur])
    {
        if (child != par)
        {
            dfs(child, cur);
            sts[cur] += sts[child];
        }
    }
    sts[cur]++;
    tout[cur] = ++timer;
}

bool is_ancestor(int u, int v)
{
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}
```

```

int lca(int u, int v)
{
    if (is_ancestor(u, v))
        return u;
    if (is_ancestor(v, u))
        return v;

    for (int i = 25; i >= 0; --i) {
        if (!is_ancestor(up[u][i], v))
            u = up[u][i];
    }

    return up[u][0];
}

int getDis(int u, int v){
    int parr = lca(u, v);
    int dis1 = abs(dep[u] - dep[parr]) + 1;
    int dis2 = abs(dep[v] - dep[parr]);
    return dis1 + dis2 - 1; // -1 to exclude single node to
        get dis
}

struct centroid {
    vector<vector<int> > edges;
    vector<bool> vis;
    vector<int> par;
    vector<int> sz;
    int n;
    void init(int s) {
        n = s;
        edges = adj;
        vis = vector<bool>(n + 1, 0);
        par = vector<int>(n + 1);
        sz = vector<int>(n + 1);
    }
    void edge(int a, int b) {
        edges[a].push_back(b);
        edges[b].push_back(a);
    }
    int find_size(int v, int p = -1) {
        if (vis[v]) return 0;
        sz[v] = 1;
        for (int x: edges[v]) {
            if (x != p) {
                sz[v] += find_size(x, v);
            }
        }
        return sz[v];
    }
    int find_centroid(int v, int p, int n) {
        for (int x: edges[v]) {

```

```

            if (x != p) {
                if (!vis[x] && sz[x] > n / 2) {
                    return find_centroid(x, v, n);
                }
            }
        }
        return v;
    }
    void init_centroid(int v, int p = -1) {
        find_size(v);
        int c = find_centroid(v, -1, sz[v]);
        vis[c] = true;
        par[c] = p;
        for (int x: edges[c]) {
            if (!vis[x]) {
                init_centroid(x, c);
            }
        }
    }
};

int n, m, k, q, l, r, x, y, z;
int a[1000005];
int b[1000005];
int c[1000005];
string s, t;
int ans = 0;
centroid ct;
int best[1000005];
void update(int v) {
    best[v] = 0;
    int u = v;
    while (ct.par[u] != -1) {
        u = ct.par[u];
        best[u] = min(best[u], getDis(u, v));
    }
}

int query(int v) {
    int ans = best[v];
    int u = v;
    while (ct.par[u] != -1) {
        u = ct.par[u];
        ans = min(ans, best[u] + getDis(u, v));
    }
    return ans;
}

void solve()
{
    int n, q; cin >> n >> q;
    up.assign(n + 1, vector<int>(26, 0));
    adj = vector<vector<int> >(n + 1);

```

```

    tin.assign(n + 1, 0);
    tout.assign(n + 1, 0);
    sts.assign(n + 1, 0);
    dep.assign(n + 1, 0);
    for (int i = 0; i < n - 1; i++) {
        int u, v; cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    for (int i = 1; i <= n; i++) best[i] = inf; // this means
        the closest red node to node i
    dfs(1);
    ct.init(n);
    ct.init_centroid(1);
    update(1); // node 1 is initially painted red
    while (q--) {
        int op, node; cin >> op >> node;
        if (op == 1) {
            update(node);
        }
        else {
            cout << query(node) << endl;
        }
    }
}

```

2.15 kosarajus

```

int scc = 0;
stack<int> st;
vector<int> vis(sz), adj[sz], adjT[sz];
void dfs(int node) {
    vis[node] = 1;
    for (auto it: adj[node]) {
        if (!vis[it]) dfs(it);
    }
    st.push(node);
}

void dfsT(int node) {
    vis[node] = 1;
    for (auto it: adjT[node]) {
        if (!vis[it]) dfsT(it);
    }
}

void kosaRajus(int n) {
    for (int i = 1; i <= n; i++) {
        if (!vis[i]) dfs(i);
    }
    for (int i = 1; i <= n; i++) {

```

```

    for(auto it: adj[i]){
        adjT[it].push_back(i);
    }
}
vis.assign(n + 1, 0);
while(!st.empty()){
    int node = st.top();
    st.pop();
    if(!vis[node]){
        dfsT(node);
        scc++;
    }
}
}
}

```

2.16 mosAlgo

```

int blk = 1;
bool cmp(pair<pair<int, int>, int> p1, pair<pair<int, int>,
int> p2){
    if((p1.first.first / blk) != (p2.first.first / blk))
        return p1.first.first < p2.first.first;
    return p1.first.second < p2.first.second; // ekhane '<='
        dile runtime error khai
}
void solve()
{
    int n;
    blk = sqrt(n);
    vector<pair<pair<int, int>, int> > qry;
    for(int i = 0; i < q; i++){
        int l, r; cin >> l >> r;
        qry.push_back({l, r}, i);
    }
    sort(all(qry), cmp);
    int lf = 1, rg = 1;
    for(int i = 0; i < q; i++){
        int l = qry[i].first.first, r = qry[i].first.second,
            ind = qry[i].second;
        while(rg <= r){
            rg++;
        }
        while(rg > r + 1){
            rg--;
        }
        while(lf < l){
            lf++;
        }
        while(lf >= l + 1){

```

```

            lf--;
        }
        ansVec[ind] = ans...;
    }
    for(auto it: ansVec){
        cout << it << nl;
    }
}

```

2.17 palindromicTree

```

const int N = 10001, mod = 998244353;
const double eps = 1e-9;
string s;
struct palindromic_tree{
    struct node {
        int len, link;
        array<int, 2> next;
        node(int l = 0) : len(l), link(0) {
            next.fill(-1);
        }
    };
    vector<node> t;
    int last;
    palindromic_tree() {
        t.clear();
    }
    t.reserve(s.size() + 3);
    t.push_back(node(-1));
    t.push_back(node(0));
    t[0].link = 0;
    t[1].link = 0;
    last = 1;
}
int getSufflink(int u, int i) {
    while(1) {
        int idx = i - 1 - t[u].len;
        if(idx >= 0 && s[idx] == s[i]) return u;
        u = t[u].link;
    }
}
int add(int i) {
    int u = getSufflink(last, i);
    int c = s[i] - 'a';
    if(t[u].next[c] != -1) {
        last = t[u].next[c];
        return 0;
    }
    node nv(t[u].len + 2);
    t.push_back(nv);

```

```

    int v = t.size() - 1;
    t[u].next[c] = v;
    if(t[v].len == 1) {
        t[v].link = 1;
    }
    else {
        int w = t[u].link;
        w = getSufflink(w, i);
        t[v].link = t[w].next[c];
    }
    last = v;
    return 1;
}
};
void solve() {
    cin >> s;
    int n = s.size();
    palindromic_tree tree;
    int pre = 0;
    string out;
    out.reserve(n);
    for (int i = 0; i < n; i++) {
        tree.add(i);
        // tree.t.size() - 2 = number of non-root nodes =
        // total distinct palindromes found so far
        int cur = tree.t.size() - 2;
        out.push_back('0' + (cur - pre));
        pre = cur;
    }
    cout << out << '\n';
}

```

2.18 persistentSegTree

```

const int N = 1e5 + 9;
struct PST {
#define lc t[cur].l
#define rc t[cur].r
    struct node {
        int l = 0, r = 0, val = 0;
    } t[20 * N];
    int T = 0;
    int build(int b, int e) {
        int cur = ++T;
        if(b == e) return cur;
        int mid = b + e >> 1;
        lc = build(b, mid);
        rc = build(mid + 1, e);
        t[cur].val = t[lc].val + t[rc].val;
    }

```

```

    return cur;
}
int upd(int pre, int b, int e, int i, int v) {
    int cur = ++T;
    t[cur] = t[pre];
    if(b == e) {
        t[cur].val += v;
        return cur;
    }
    int mid = b + e >> 1;
    if(i <= mid) {
        rc = t[pre].r;
        lc = upd(t[pre].l, b, mid, i, v);
    } else {
        lc = t[pre].l;
        rc = upd(t[pre].r, mid + 1, e, i, v);
    }
    t[cur].val = t[lc].val + t[rc].val;
    return cur;
}
int query(int pre, int cur, int b, int e, int k) {
    if(b == e) return b;
    int cnt = t[lc].val - t[t[pre].l].val;
    int mid = b + e >> 1;
    if(cnt >= k) return query(t[pre].l, lc, b, mid, k);
    else return query(t[pre].r, rc, mid + 1, e, k - cnt);
}
} t;

//the code returns k-th number in a range l to r if the
//range were sorted
int V[N], root[N], a[N];
int32_t main() {
    map<int, int> mp;
    int n, q;
    cin >> n >> q;
    for(int i = 1; i <= n; i++) cin >> a[i], mp[a[i]];
    int c = 0;
    for(auto x : mp) mp[x.first] = ++c, V[c] = x.first;
    root[0] = t.build(1, n);
    for(int i = 1; i <= n; i++) {
        root[i] = t.upd(root[i - 1], 1, n, mp[a[i]], 1);
    }
    while(q--) {
        int l, r, k;
        cin >> l >> r >> k;
        cout << V[t.query(root[l - 1], root[r], 1, n, k)] << '\n';
    }
    return 0;
}

```

```

}

```

2.19 virtualTree

```

const int N = 100005;
const int LOGN = 20;
vector<int> g[N];
vector<int> adjacency_vt[N];
int tin[N], tout[N], timer;
int up[N][LOGN];
int depth[N];
bool important[N];
int subtree_count[N];
long long ans;
int total_count;
void dfs(int u, int p, int d) {
    tin[u] = ++timer;
    depth[u] = d;
    up[u][0] = p;
    for (int i = 1; i < LOGN; ++i) {
        up[u][i] = up[up[u][i - 1]][i - 1];
    }
    for (int v : g[u]) {
        if (v != p) {
            dfs(v, u, d + 1);
        }
    }
    tout[u] = ++timer;
}
bool upper(int u, int v) {
    return tin[u] <= tin[v] && tout[u] >= tout[v];
}
int lca(int u, int v) {
    if (upper(u, v)) return u;
    if (upper(v, u)) return v;
    for (int i = LOGN - 1; i >= 0; --i) {
        if (!upper(up[u][i], v)) {
            u = up[u][i];
        }
    }
    return up[u][0];
}
bool cmp(int a, int b) {
    return tin[a] < tin[b];
}
int build_virtual_tree(vector<int>& vert) {
    sort(vert.begin(), vert.end(), cmp);
    int k = vert.size();
    for (int i = 0; i < k - 1; ++i) {

```

```

        vert.push_back(lca(vert[i], vert[i + 1]));
    }
    sort(vert.begin(), vert.end(), cmp);
    vert.erase(unique(vert.begin(), vert.end()), vert.end());
    for (int u : vert) {
        adjacency_vt[u].clear();
    }
    vector<int> stack_v;
    stack_v.push_back(vert[0]);
    for (int i = 1; i < vert.size(); ++i) {
        int u = vert[i];
        while (stack_v.size() >= 2 && !upper(stack_v.back(), u)) {
            adjacency_vt[stack_v[stack_v.size() - 2]].push_back(stack_v.back());
            stack_v.pop_back();
        }
        stack_v.push_back(u);
    }
    while (stack_v.size() >= 2) {
        adjacency_vt[stack_v[stack_v.size() - 2]].push_back(stack_v.back());
        stack_v.pop_back();
    }
    return stack_v[0];
}
void solve(int u, int p) {
    subtree_count[u] = important[u];
    for (int v : adjacency_vt[u]) {
        solve(v, u);
        subtree_count[u] += subtree_count[v];
    }
    if (p != -1) {
        int w = depth[u] - depth[p];
        ans += (long long)w * subtree_count[u] * (total_count - subtree_count[u]);
    }
}
int main() {
    int n; cin >> n;
    for (int i = 0; i < n - 1; ++i) {
        int u, v;
        cin >> u >> v;
        g[u].push_back(v);
        g[v].push_back(u);
    }
    timer = 0;
    dfs(1, 1, 0);
    int q;
    cin >> q;

```



```

while (q--) {
    int k;
    cin >> k;
    vector<int> vert(k);
    for (int i = 0; i < k; ++i) {
        cin >> vert[i];
        important[vert[i]] = true;
    }
    total_count = k;
    ans = 0;
    int vt_root = build_virtual_tree(vert);
    solve(vt_root, -1);
    cout << ans << "\n";
    for (int x : vert) {
        important[x] = false;
    }
}
return 0;

```

3 Geo

3.1 Closest Pair

```

#define x first
#define y second
long long dist2(pair<int, int> a, pair<int, int> b) {
    return 1LL * (a.x - b.x) * (a.x - b.x) + 1LL * (a.y - b.y)
        * (a.y - b.y);}
pair<int, int> closest_pair(vector<pair<int, int>> a) {
    int n = a.size();
    assert(n >= 2);
    vector<pair<pair<int, int>, int>> p(n);
    for (int i = 0; i < n; i++) p[i] = {a[i], i};
    sort(p.begin(), p.end());
    int l = 0, r = 2;
    long long ans = dist2(p[0].x, p[1].x);
    pair<int, int> ret = {p[0].y, p[1].y};
    while (r < n) {
        while (l < r && 1LL * (p[r].x.x - p[l].x.x) * (p[r].x.x - p[l].x.x) >= ans) l++;
        for (int i = l; i < r; i++) {
            long long nw = dist2(p[i].x, p[r].x);
            if (nw < ans) {
                ans = nw;
                ret = {p[i].y, p[r].y;};}
        }
        r++;
    }
    return ret;
}

```

3.2 Geometry 2D

```

#include<bits/stdc++.h>
using namespace std;
const int N = 3e5 + 9;
const double inf = 1e100;
const double PI = acos((double)-1.0);
const double eps = 1e-9;
int sign(double x) { return (x > eps) - (x < -eps); }
struct PT {
    double x, y;
    PT() { x = 0, y = 0; }
    PT(double x, double y) : x(x), y(y) {}
    PT(const PT &p) : x(p.x), y(p.y) {}
    PT operator + (const PT &a) const { return PT(x + a.x, y + a.y); }
    PT operator - (const PT &a) const { return PT(x - a.x, y - a.y); }
    PT operator * (const double a) const { return PT(x * a, y * a); }
    friend PT operator * (const double &a, const PT &b) {
        return PT(a * b.x, a * b.y); }
    PT operator / (const double a) const { return PT(x / a, y / a); }
    bool operator == (PT a) const { return sign(a.x - x) == 0
        && sign(a.y - y) == 0; }
    bool operator != (PT a) const { return !(*this == a); }
    bool operator < (PT a) const { return sign(a.x - x) == 0
        ? y < a.y : x < a.x; }
    bool operator > (PT a) const { return sign(a.x - x) == 0
        ? y > a.y : x > a.x; }
    double norm() { return sqrt(x * x + y * y); }
    double norm2() { return x * x + y * y; }
    PT perp() { return PT(-y, x); }
    double arg() { return atan2(y, x); }
    PT truncate(double r) { // returns a vector with norm r
        and having same direction
        double k = norm();
        if (!sign(k)) return *this;
        r /= k;
        return PT(x * r, y * r);
    }
};
istream &operator >> (istream &in, PT &p) { return in >> p.x
    >> p.y; }
ostream &operator << (ostream &out, PT &p) { return out << "
    (" << p.x << ", " << p.y << ")"; }
inline double dot(PT a, PT b) { return a.x * b.x + a.y * b.y
    ; }

```

```

inline double dist2(PT a, PT b) { return dot(a - b, a - b);
    }
inline double dist(PT a, PT b) { return sqrt(dot(a - b, a - b)); }
inline double cross(PT a, PT b) { return a.x * b.y - a.y * b.x; }
inline double cross2(PT a, PT b, PT c) { return cross(b - a, c - a); }
inline int orientation(PT a, PT b, PT c) { return sign(cross(b - a, c - a)); }
PT perp(PT a) { return PT(-a.y, a.x); }
PT rotateccw90(PT a) { return PT(-a.y, a.x); }
PT rotatecw90(PT a) { return PT(a.y, -a.x); }
PT rotateccw(PT a, double t) { return PT(a.x * cos(t) - a.y * sin(t), a.x * sin(t) + a.y * cos(t)); }
PT rotatecw(PT a, double t) { return PT(a.x * cos(t) + a.y * sin(t), -a.x * sin(t) + a.y * cos(t)); }
double SQ(double x) { return x * x; }
double rad_to_deg(double r) { return (r * 180.0 / PI); }
double deg_to_rad(double d) { return (d * PI / 180.0); }
// returns in rad
double get_angle(PT a, PT b) {
    double costheta = dot(a, b) / a.norm() / b.norm();
    return acos(max((double)-1.0, min((double)1.0, costheta)));
}
struct line {
    PT a, b; // goes through points a and b
    PT v; double c; //line form: direction vec [cross] (x, y) = c
    line() {}
    //direction vector v and offset c
    line(PT v, double c) : v(v), c(c) {
        auto p = get_points();
        a = p.first; b = p.second;
    }
    // equation ax + by + c = 0
    line(double _a, double _b, double _c) : v({_b, -_a}), c(-_c) {}
    auto p = get_points();
    a = p.first; b = p.second;
}
// goes through points p and q
line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q) {}
pair<PT, PT> get_points() { //extract any two points from this line
    PT p, q; double a = -v.y, b = v.x; // ax + by = c
    if (sign(a) == 0) {
        p = PT(0, c / b);
        q = PT(1, c / b);
    }
}

```

```

}
else if (sign(b) == 0) {
    p = PT(c / a, 0);
    q = PT(c / a, 1);
}
else {
    p = PT(0, c / b);
    q = PT(1, (c - a) / b);
}
return {p, q};
}
// ax + by + c = 0
array<double, 3> get_abc() {
    double a = -v.y, b = v.x;
    return {a, b, -c};
}
// 1 if on the left, -1 if on the right, 0 if on the line
int side(PT p) { return sign(cross(v, p) - c); }
// line that is perpendicular to this and goes through
// point p
line perpendicular_through(PT p) { return {p, p + perp(v)}; }
// translate the line by vector t i.e. shifting it by
// vector t
line translate(PT t) { return {v, c + cross(v, t)}; }
// compare two points by their orthogonal projection on
// this line
// a projection point comes before another if it comes
// first according to vector v
bool cmp_by_projection(PT p, PT q) { return dot(v, p) <
    dot(v, q); }
line shift_left(double d) {
    PT z = v.perp().truncate(d);
    return line(a + z, b + z);
}
};
// minimum distance from point c to line through a and b
double dist_from_point_to_line(PT a, PT b, PT c) {
    return fabs(cross(b - a, c - a) / (b - a).norm());
}
// returns true if point p is on line segment ab
bool is_point_on_seg(PT a, PT b, PT p) {
    if (fabs(cross(p - b, a - b)) < eps) {
        if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x)
            + eps) return false;
        if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y)
            + eps) return false;
        return true;
    }
    return false;
}

```

```

}
// minimum distance point from point c to segment ab that
// lies on segment ab
PT project_from_point_to_seg(PT a, PT b, PT c) {
    double r = dist2(a, b);
    if (sign(r) == 0) return a;
    r = dot(c - a, b - a) / r;
    if (r < 0) return a;
    if (r > 1) return b;
    return a + (b - a) * r;
}
// minimum distance from point c to segment ab
double dist_from_point_to_seg(PT a, PT b, PT c) {
    return dist(c, project_from_point_to_seg(a, b, c));
}
// 0 if not parallel, 1 if parallel, 2 if collinear
int is_parallel(PT a, PT b, PT c, PT d) {
    double k = fabs(cross(b - a, d - c));
    if (k < eps) {
        if (fabs(cross(a - b, a - c)) < eps && fabs(cross(c -
            d, c - a)) < eps) return 2;
        else return 1;
    }
    else return 0;
}
// 1 if point is ccw to the line, 2 if point is cw to the
// line, 3 if point is on the line
int point_line_relation(PT a, PT b, PT p) {
    int c = sign(cross(p - a, b - a));
    if (c < 0) return 1;
    if (c > 0) return 2;
    return 3;
}
// intersection point between ab and cd assuming unique
// intersection exists
bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans)
{
    double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
    double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
    double det = a1 * b2 - a2 * b1;
    if (det == 0) return 0;
    ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2) /
        det);
    return 1;
}
// intersection point between segment ab and segment cd
// assuming unique intersection exists
bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans) {
    double oa = cross2(c, d, a), ob = cross2(c, d, b);
    double oc = cross2(a, b, c), od = cross2(a, b, d);
    if (oa * ob < 0 && oc * od < 0) {
        ans = (a * ob - b * oa) / (ob - oa);
        return 1;
    }
    else return 0;
}

```

```

if (oa * ob < 0 && oc * od < 0) {
    ans = (a * ob - b * oa) / (ob - oa);
    return 1;
}
else return 0;
}
// mychanges
// even if no unique intersection, meaning in the same
// direction
bool seg_seg_intersection(PT a, PT b, PT c, PT d) {
    double oa = cross2(c, d, a), ob = cross2(c, d, b);
    double oc = cross2(a, b, c), od = cross2(a, b, d);
    if (oa * ob < 0 && oc * od < 0) {
        return true;
    }
    // checking orientation for very precise calculation
    else if (oa == 0 && is_point_on_seg(c, d, a)) return true;
    else if (ob == 0 && is_point_on_seg(c, d, b)) return true;
    else if (oc == 0 && is_point_on_seg(a, b, c)) return true;
    else if (od == 0 && is_point_on_seg(a, b, d)) return true;
    else return false;
}
// intersection between segment ab and line cd
// 0 if do not intersect, 1 if proper intersect, 2 if
// segment intersect
int seg_line_relation(PT a, PT b, PT c, PT d) {
    double p = cross2(c, d, a);
    double q = cross2(c, d, b);
    if (sign(p) == 0 && sign(q) == 0) return 2;
    else if (p * q < 0) return 1;
    else return 0;
}
// intersection between segment ab and line cd assuming
// unique intersection exists
bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans)
{
    bool k = seg_line_relation(a, b, c, d);
    assert(k != 2);
    if (k) line_line_intersection(a, b, c, d, ans);
    return k;
}
// minimum distance from segment ab to segment cd
double dist_from_seg_to_seg(PT a, PT b, PT c, PT d) {
    PT dummy;
    if (seg_seg_intersection(a, b, c, d, dummy)) return 0.0;
    else return min({dist_from_point_to_seg(a, b, c),
        dist_from_point_to_seg(a, b, d),
        dist_from_point_to_seg(c, d, a),
        dist_from_point_to_seg(c, d, b)});
}
}

```

```

struct circle {
    PT p; double r;
    circle() {}
    circle(PT _p, double _r): p(_p), r(_r) {};
    // center (x, y) and radius r
    circle(double x, double y, double _r): p(PT(x, y)), r(_r)
    {};
    // circumcircle of a triangle
    // the three points must be unique
    circle(PT a, PT b, PT c) {
        b = (a + b) * 0.5;
        c = (a + c) * 0.5;
        line_line_intersection(b, b + rotatecw90(a - b), c, c
            + rotatecw90(a - c), p);
        r = dist(a, p);
    }
    // inscribed circle of a triangle
    // pass a bool just to differentiate from circumcircle
    circle(PT a, PT b, PT c, bool t) {
        line u, v;
        double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y
            - a.y, c.x - a.x);
        u.a = a;
        u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
        v.a = b;
        m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y,
            c.x - b.x);
        v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
        line_line_intersection(u.a, u.b, v.a, v.b, p);
        r = dist_from_point_to_seg(a, b, p);
    }
    bool operator == (circle v) { return p == v.p && sign(r -
        v.r) == 0; }
    double area() { return PI * r * r; }
    double circumference() { return 2.0 * PI * r; }
};
//0 if outside, 1 if on circumference, 2 if inside circle
int circle_point_relation(PT p, double r, PT b) {
    double d = dist(p, b);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}
// 0 if outside, 1 if on circumference, 2 if inside circle
int circle_line_relation(PT p, double r, PT a, PT b) {
    double d = dist_from_point_to_line(a, b, p);
    if (sign(d - r) < 0) return 2;
    if (sign(d - r) == 0) return 1;
    return 0;
}

//compute intersection of line through points a and b with
//circle centered at c with radius r > 0
vector<PT> circle_line_intersection(PT c, double r, PT a, PT
    b) {
    vector<PT> ret;
    b = b - a; a = a - c;
    double A = dot(b, b), B = dot(a, b);
    double C = dot(a, a) - r * r, D = B * B - A * C;
    if (D < -eps) return ret;
    ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
    if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) / A
        );
    return ret;
}
//5 - outside and do not intersect
//4 - intersect outside in one point
//3 - intersect in 2 points
//2 - intersect inside in one point
//1 - inside and do not intersect
int circle_circle_relation(PT a, double r, PT b, double R) {
    double d = dist(a, b);
    if (sign(d - r - R) > 0) return 5;
    if (sign(d - r - R) == 0) return 4;
    double l = fabs(r - R);
    if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
    if (sign(d - l) == 0) return 2;
    if (sign(d - l) < 0) return 1;
    assert(0); return -1;
}
vector<PT> circle_circle_intersection(PT a, double r, PT b,
    double R) {
    if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
    vector<PT> ret;
    double d = sqrt(dist2(a, b));
    if (d > r + R || d + min(r, R) < max(r, R)) return ret;
    double x = (d * d - R * R + r * r) / (2 * d);
    double y = sqrt(r * r - x * x);
    PT v = (b - a) / d;
    ret.push_back(a + v * x + rotateccw90(v) * y);
    if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y);
    return ret;
}
// returns two circle c1, c2 through points a, b and of
// radius r
// 0 if there is no such circle, 1 if one circle, 2 if two
// circle
int get_circle(PT a, PT b, double r, circle &c1, circle &c2)
    {
        vector<PT> v = circle_circle_intersection(a, r, b, r);
        int t = v.size();
        if (!t) return 0;
        c1.p = v[0], c1.r = r;
        if (t == 2) c2.p = v[1], c2.r = r;
        return t;
    }
// returns area of intersection between two circles
double circle_circle_area(PT a, double r1, PT b, double r2)
    {
        double d = (a - b).norm();
        if (r1 + r2 < d + eps) return 0;
        if (r1 + d < r2 + eps) return PI * r1 * r1;
        if (r2 + d < r1 + eps) return PI * r2 * r2;
        double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 *
            r1 * d));
        theta_2 = acos((r2 * r2 + d * d - r1 * r1) / (2 * r2 * d));
        return r1 * r1 * (theta_1 - sin(2 * theta_1)/2.) + r2 *
            r2 * (theta_2 - sin(2 * theta_2)/2.);
    }
double area_of_triangle(PT a, PT b, PT c) {
    return fabs(cross(b - a, c - a) * 0.5);
}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly
// outside
int is_point_in_triangle(PT a, PT b, PT c, PT p) {
    if (sign(cross(b - a, c - a)) < 0) swap(b, c);
    int c1 = sign(cross(b - a, p - a));
    int c2 = sign(cross(c - b, p - b));
    int c3 = sign(cross(a - c, p - c));
    if (c1 < 0 || c2 < 0 || c3 < 0) return 1;
    if (c1 + c2 + c3 != 3) return 0;
    return -1;
}
double area(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1)
        % n]);
    return fabs(ans) * 0.5;
}
// 0 if cw, 1 if ccw
bool get_direction(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1)
        % n]);
    if (sign(ans) > 0) return 1;
    return 0;
}
// returns the convex hull of the set of points p
vector<PT> convex_hull(vector<PT> &p) {
    if (p.size() <= 1) return p;

```

```

vector<PT> v = p;
sort(v.begin(), v.end());
vector<PT> up, dn;
for (auto& p : v) {
    while (up.size() > 1 && orientation(up[up.size() - 2], up.back(), p) >= 0) {
        up.pop_back();
    }
    while (dn.size() > 1 && orientation(dn[dn.size() - 2], dn.back(), p) <= 0) {
        dn.pop_back();
    }
    up.push_back(p);
    dn.push_back(p);
}
v = dn;
if (v.size() > 1) v.pop_back();
reverse(up.begin(), up.end());
up.pop_back();
for (auto& p : up) {
    v.push_back(p);
}
if (v.size() == 2 && v[0] == v[1]) v.pop_back();
return v;
}

//checks if convex or not
bool is_convex(vector<PT> &p) {
    bool s[3]; s[0] = s[1] = s[2] = 0;
    int n = p.size();
    for (int i = 0; i < n; i++) {
        int j = (i + 1) % n;
        int k = (j + 1) % n;
        s[sign(cross(p[j] - p[i], p[k] - p[i])) + 1] = 1;
        if (s[0] && s[2]) return 0;
    }
    return 1;
}

// -1 if strictly inside, 0 if on the polygon, 1 if strictly outside
// it must be strictly convex, otherwise make it strictly convex first
int is_point_in_convex(vector<PT> &p, const PT& x) { // O(log n)
    int n = p.size(); assert(n >= 3);
    int a = orientation(p[0], p[1], x), b = orientation(p[0], p[n - 1], x);
    if (a < 0 || b > 0) return 1;
    int l = 1, r = n - 1;
    while (l + 1 < r) {
        int mid = l + r >> 1;

```

```

        if (orientation(p[0], p[mid], x) >= 0) l = mid;
        else r = mid;
    }
    int k = orientation(p[l], p[r], x);
    if (k <= 0) return -k;
    if (l == 1 && a == 0) return 0;
    if (r == n - 1 && b == 0) return 0;
    return -1;
}

bool is_point_on_polygon(vector<PT> &p, const PT& z) {
    int n = p.size();
    for (int i = 0; i < n; i++) {
        if (is_point_on_seg(p[i], p[(i + 1) % n], z)) return 1;
    }
    return 0;
}

// returns 1e9 if the point is on the polygon
int winding_number(vector<PT> &p, const PT& z) { // O(n)
    if (is_point_on_polygon(p, z)) return 1e9;
    int n = p.size(), ans = 0;
    for (int i = 0; i < n; ++i) {
        int j = (i + 1) % n;
        bool below = p[i].y < z.y;
        if (below != (p[j].y < z.y)) {
            auto orient = orientation(z, p[j], p[i]);
            if (orient == 0) return 0;
            if (below == (orient > 0)) ans += below ? 1 : -1;
        }
    }
    return ans;
}

// -1 if strictly inside, 0 if on the polygon, 1 if strictly outside
int is_point_in_polygon(vector<PT> &p, const PT& z) { // O(n)
    int k = winding_number(p, z);
    return k == 1e9 ? 0 : k == 0 ? 1 : -1;
}

// id of the vertex having maximum dot product with z
// polygon must need to be convex
// top - upper right vertex
// for minimum dot product negate z and return -dot(z, p[id])
int extreme_vertex(vector<PT> &p, const PT &z, const int top) { // O(log n)
    int n = p.size();
    if (n == 1) return 0;
    double ans = dot(p[0], z); int id = 0;
    if (dot(p[top], z) > ans) ans = dot(p[top], z), id = top;
    int l = 1, r = top - 1;

```

```

    while (l < r) {
        int mid = l + r >> 1;
        if (dot(p[mid + 1], z) >= dot(p[mid], z)) l = mid + 1;
        else r = mid;
    }
    if (dot(p[l], z) > ans) ans = dot(p[l], z), id = l;
    l = top + 1, r = n - 1;
    while (l < r) {
        int mid = l + r >> 1;
        if (dot(p[(mid + 1) % n], z) >= dot(p[mid], z)) l = mid + 1;
        else r = mid;
    }
    l %= n;
    if (dot(p[l], z) > ans) ans = dot(p[l], z), id = l;
    return id;
}

pair<PT, int> point_poly_tangent(vector<PT> &p, PT Q, int dir, int l, int r) {
    while (r - l > 1) {
        int mid = (l + r) >> 1;
        bool pvs = orientation(Q, p[mid], p[mid - 1]) != -dir;
        bool nxt = orientation(Q, p[mid], p[mid + 1]) != -dir;
        if (pvs && nxt) return {p[mid], mid};
        if (!pvs || !nxt) {
            auto p1 = point_poly_tangent(p, Q, dir, mid + 1, r);
            auto p2 = point_poly_tangent(p, Q, dir, l, mid - 1);
            return orientation(Q, p1.first, p2.first) == dir ? p1 : p2;
        }
    }
    if (!pvs) {
        if (orientation(Q, p[mid], p[l]) == dir) r = mid - 1;
        else if (orientation(Q, p[l], p[r]) == dir) r = mid - 1;
        else l = mid + 1;
    }
    if (!nxt) {
        if (orientation(Q, p[mid], p[l]) == dir) l = mid + 1;
        else if (orientation(Q, p[l], p[r]) == dir) r = mid - 1;
        else l = mid + 1;
    }
}

```

```

pair<PT, int> ret = {p[l], l};
for (int i = l + 1; i <= r; i++) ret = orientation(Q, ret
    .first, p[i]) != dir ? make_pair(p[i], i) : ret;
return ret;
}
// (ccw, cw) tangents from a point that is outside this
// convex polygon
// returns indexes of the points
// ccw means the tangent from Q to that point is in the same
// direction as the polygon ccw direction
pair<int, int> tangents_from_point_to_polygon(vector<PT> &p,
    PT Q){
    int ccw = point_poly_tangent(p, Q, 1, 0, (int)p.size() -
        1).second;
    int cw = point_poly_tangent(p, Q, -1, 0, (int)p.size() -
        1).second;
    return make_pair(ccw, cw);
}
// minimum distance from a point to a convex polygon
// it assumes point lie strictly outside the polygon
double dist_from_point_to_polygon(vector<PT> &p, PT z) {
    double ans = inf;
    int n = p.size();
    if (n <= 3) {
        for(int i = 0; i < n; i++) ans = min(ans,
            dist_from_point_to_seg(p[i], p[(i + 1) % n], z));
        return ans;
    }
    auto [r, l] = tangents_from_point_to_polygon(p, z);
    if(l > r) r += n;
    while (l < r) {
        int mid = (l + r) >> 1;
        double left = dist2(p[mid % n], z), right = dist2(p[(
            mid + 1) % n], z);
        ans = min({ans, left, right});
        if(left < right) r = mid;
        else l = mid + 1;
    }
    ans = sqrt(ans);
    ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l + 1)
        % n], z));
    ans = min(ans, dist_from_point_to_seg(p[l % n], p[(l - 1
        + n) % n], z));
    return ans;
}
// minimum distance from convex polygon p to line ab
// returns 0 is it intersects with the polygon
// top - upper right vertex

```

```

double dist_from_polygon_to_line(vector<PT> &p, PT a, PT b,
    int top) { //O(log n)
    PT orth = (b - a).perp();
    if (orientation(a, b, p[0]) > 0) orth = (a - b).perp();
    int id = extreme_vertex(p, orth, top);
    if (dot(p[id] - a, orth) > 0) return 0.0; //if orth and a
    // are in the same half of the line, then poly and line
    // intersects
    return dist_from_point_to_line(a, b, p[id]); //does not
    // intersect
}
// minimum distance from a convex polygon to another convex
// polygon
// the polygon doesnot overlap or touch
// tested in https://toph.co/p/the-wall
double dist_from_polygon_to_polygon(vector<PT> &p1, vector<
    PT> &p2) { // O(n log n)
    double ans = inf;
    for (int i = 0; i < p1.size(); i++) {
        ans = min(ans, dist_from_point_to_polygon(p2, p1[i]));
    }
    for (int i = 0; i < p2.size(); i++) {
        ans = min(ans, dist_from_point_to_polygon(p1, p2[i]));
    }
    return ans;
}
// maximum distance from a convex polygon to another convex
// polygon
double maximum_dist_from_polygon_to_polygon(vector<PT> &u,
    vector<PT> &v){ //O(n)
    int n = (int)u.size(), m = (int)v.size();
    double ans = 0;
    if (n < 3 || m < 3) {
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) ans = max(ans, dist2(
                u[i], v[j]));
        }
        return sqrt(ans);
    }
    if (u[0].x > v[0].x) swap(n, m), swap(u, v);
    int i = 0, j = 0, step = n + m + 10;
    while (j + 1 < m && v[j].x < v[j + 1].x) j++;
    while (step-- > 0) {
        if (cross(u[(i + 1) % n] - u[i], v[(j + 1) % m] - v[j])
            >= 0) j = (j + 1) % m;
        else i = (i + 1) % n;
        ans = max(ans, dist2(u[i], v[j]));
    }
}

```

```

    return sqrt(ans);
}
// mychanges
// did not test but should work
double polygon_area(vector<PT> &v){
    double area = 0;
    int n = v.size();
    for(int i = 0; i < n; i++){
        PT a = v[i], b = v[(i + 1) % n];
        int x1 = a.x, y1 = a.y;
        int x2 = b.x, y2 = b.y;
        area += (x1 * y2 - x2 * y1);
    }
    area = abs(area);
    area = area / 2.0; // this can give precision errors if
    // value is so much beacuse double have big value
    // precision errors
    return area;
}
// mychanges
// exist named is_point_in_polygon
int position_of_a_point_in_a_polygon(vector<PT> &v, PT c){
    double xInf = inf, yInf = c.y + 1;
    PT d(xInf, yInf);
    int n = v.size();

    int cnt = 0;
    for(int i = 0; i < n; i++){
        PT a = v[i], b = v[(i + 1) % n];
        int val = seg_seg_intersection(a, b, c, d);
        if(val == 1) cnt++;
        else if(val == 2) return 0; // BOUNDARY
    }
    if(cnt % 2) return 1; // INSIDE
    else return -1; // OUTSIDE
}

int getSign(int x1, int y1, int x2, int y2, int x3, int y3){
    // int direct = x1 * y2 + x2 * y3 + x3 * y1 - x1 * y3 -
    // x2 * y1 - x3 * y2;
    int direct = (y2 - y1) * (x3 - x2) - (y3 - y2) * (x2 - x1
        );
    if(direct > 0) return 1; // clockWise
    if(direct < 0) return 2; // counter - ClockWise
    if(direct == 0) return 0; // cointinear
}
// Function to find orientation of ordered triplet (x1,y1),
// (x2,y2), (x3,y3)
// Returns: 0 -> collinear, 1 -> clockwise, 2 -> counter-
// clockwise

```

```
int ori(int x1, int y1, int x2, int y2, int x3, int y3) {
    int val = (y2 - y1) * (x3 - x2) - (x2 - x1) * (y3 - y2);
    if (val == 0) return 1; // clockwise
    else if (val < 0) return 2; // counter-clockwise
    return 0; // collinear
}
```

4 Graph Trees

4.1 Cycle Detection

```
const int N = 5e5 + 9;
vector<pair<int, int>> g[N];
int vis[N], par[N], e_id[N];
vector<int> cycle; // simple cycle, contains edge ids
bool dfs(int u) {
    if (!cycle.empty()) return 1;
    vis[u] = 1;
    for (auto [v, id] : g[u]) {
        if (v != par[u]) {
            if (vis[v] == 0) {
                par[v] = u;
                e_id[v] = id;
                if (dfs(v)) return 1;
            } else if (vis[v] == 1) {
                cycle.push_back(id); // cycle here
                for (int x = u; x != v; x = par[x]) {
                    cycle.push_back(e_id[x]);
                }
                return 1;
            }
        }
    }
    vis[u] = 2;
    return 0;
}
```

4.2 DJ xtra

```
const int N = int(2e5)+3, inf = int(1e16);
vector<pair<int, int>> g[N]; // in graph weight first &
    neighbour second
vector<int> dis(N, inf);
void SP(int source) {
    priority_queue<pair<int, int>, vector<pair<int, int>>,
        greater<pair<int, int>>> pq;
```

```
dis[source] = 0; pq.push({0, source});
par[source] = -1; // if parent is necessary
while (!pq.empty()) {
    int v = pq.top().second, wt = pq.top().first; pq.pop();
    if (dis[v] < wt) continue;
    for (auto &it : g[v]) {
        int child_v = it.second, child_wt = it.first;
        if (dis[v] + child_wt < dis[child_v]) {
            par[child_v] = v; // if parent is necessary
            dis[child_v] = dis[v] + child_wt;
            pq.push({dis[child_v], child_v});
        }
    }
}
```

4.3 LCA

```
const int N = 3e5 + 9, LG = 18;
vector<int> g[N];
int par[N][LG + 1], dep[N], sz[N];
void dfs(int u, int p = 0) {
    par[u][0] = p;
    dep[u] = dep[p] + 1;
    sz[u] = 1;
    for (int i = 1; i <= LG; i++) par[u][i] = par[par[u][i - 1]][i - 1];
    for (auto v : g[u]) if (v != p) {
        dfs(v, u);
        sz[u] += sz[v];
    }
}
int lca(int u, int v) {
    if (dep[u] < dep[v]) swap(u, v);
    for (int k = LG; k >= 0; k--) if (dep[par[u][k]] >= dep[v])
        u = par[u][k];
    if (u == v) return u;
    for (int k = LG; k >= 0; k--) if (par[u][k] != par[v][k])
        u = par[u][k], v = par[v][k];
    return par[u][0];
}
int kth(int u, int k) {
    assert(k >= 0);
    for (int i = 0; i <= LG; i++) if (k & (1 << i)) u = par[u][i];
    return u;
}
int dist(int u, int v) {
    int l = lca(u, v);
```

```
    return dep[u] + dep[v] - (dep[l] << 1);
}
//kth node from u to v, 0th node is u
int go(int u, int v, int k) {
    int l = lca(u, v);
    int d = dep[u] + dep[v] - (dep[l] << 1);
    assert(k <= d);
    if (dep[l] + k <= dep[u]) return kth(u, k);
    k -= dep[u] - dep[l];
    return kth(v, dep[v] - dep[l] - k);
}
```

4.4 MaxFlow

```
const int64_t inf = 1LL << 60;
struct Dinic {
    struct Edge {
        int to, rev; int64_t flow, w; int id;
    };
    int n, s, t, mxid; vector<int> d, done;
    vector<int64_t> flow_through; vector<vector<Edge>> g;
    Dinic() {}
    Dinic(int _n) {
        n = _n + 10;
        mxid = 0;
        g.resize(n);
    }
    void add_edge(int u, int v, int64_t w, int id = -1) {
        Edge a = {v, (int)g[v].size(), 0, w, id};
        Edge b = {u, (int)g[u].size(), 0, 0, -2}; // cap(b) = w
        for bidirectional
            g[u].emplace_back(a);
            g[v].emplace_back(b);
            if (id >= 0) mxid = max(mxid, id);
    }
    bool bfs() {
        d.assign(n, -1); d[s] = 0;
        queue<int> q; q.push(s);
        while (!q.empty()) {
            int u = q.front();
            q.pop();
            for (auto &e : g[u]) {
                int v = e.to;
                if (d[v] == -1 && e.flow < e.w) {
                    d[v] = d[u] + 1;
                    q.push(v);
                }
            }
        }
    }
```



```

    return d[t] != -1;
}
int64_t dfs(int u, int64_t flow) {
    if (u == t) return flow;
    for (int &i = done[u]; i < (int)g[u].size(); i++) {
        Edge &e = g[u][i];
        if (e.w <= e.flow) continue;
        int v = e.to;
        if (d[v] == d[u] + 1) {
            int64_t nw = dfs(v, min(flow, e.w - e.flow));
            if (nw > 0) {
                e.flow += nw;
                g[v][e.rev].flow -= nw;
                return nw;
            }
        }
    }
    return 0;
}
int64_t max_flow(int _s, int _t) {
    s = _s;
    t = _t;
    int64_t flow = 0;
    while (bfs()) {
        done.assign(n, 0);
        while (int64_t nw = dfs(s, inf)) {
            flow += nw;
        }
    }
    flow_through.assign(mxid+10, int64_t(0));
    for (int i = 0; i < n; i++) {
        for (auto &e : g[i]) {
            if (e.id >= 0) {
                flow_through[e.id] = e.flow;
            }
        }
    }
    return flow;
}
};

```

4.5 Min Cost Flow

```

const int64_t inf = int(1e18);
struct MCF{
    struct Edge {
        int to, rev; int64_t cap, cost, flow = 0;
    };
    int n; vector<vector<Edge>> g;

```

```

    vector<int64_t> h, dist; vector<int> prevv, preve;
    MCF(int _n = 0) {
        init(_n);
    }
    void init(int _n) {
        n = _n;
        g.assign(n, {});
        h.assign(n, 0);
        dist.assign(n, 0);
        prevv.assign(n, 0);
        preve.assign(n, 0);
    }
    void add(int u, int v, int64_t cap, int64_t cost) {
        g[u].push_back({v, (int)g[v].size(), cap, cost});
        g[v].push_back({u, (int)g[u].size() - 1, 0, -cost});
    }
    pair<int64_t, int64_t> flow(int s, int t, int64_t need =
        inf) {
        pair<int64_t, int64_t> res = {0, 0};
        fill(h.begin(), h.end(), 0LL);
        while (need > 0) {
            priority_queue<pair<int64_t, int>, vector<pair<
                int64_t, int>>, greater<pair<int64_t, int>>> pq
                ;
            fill(dist.begin(), dist.end(), inf);
            dist[s] = 0;
            pq.emplace(0, s);
            while (!pq.empty()) {
                auto [d, v] = pq.top();
                pq.pop();
                if (dist[v] < d) continue;
                for (int i = 0; i < (int)g[v].size(); ++i) {
                    Edge &e = g[v][i];
                    if (e.cap > e.flow) {
                        int64_t nc = d + e.cost + h[v] - h[e.to];
                        if (dist[e.to] > nc) {
                            dist[e.to] = nc;
                            prevv[e.to] = v;
                            preve[e.to] = i;
                            pq.emplace(nc, e.to);
                        }
                    }
                }
            }
            if (dist[t] == inf) break;
            for (int i = 0; i < n; ++i) {
                if (dist[i] < inf) {
                    h[i] += dist[i];
                }
            }
        }
        return res;
    }
};

```

```

    int64_t push = need;
    for (int v = t; v != s; v = prevv[v]) {
        push = min(push, g[prevv[v]][preve[v]].cap - g[
            prevv[v]][preve[v]].flow);
    }
    need -= push;
    res.first += push;
    res.second += push * h[t];
    for (int v = t; v != s; v = prevv[v]) {
        Edge &e = g[prevv[v]][preve[v]];
        e.flow += push;
        g[e.to][e.rev].flow -= push;
    }
}
return res;
}
void clear_flow() {
    for (auto &vec : g) {
        for (auto &e : vec) {
            e.flow = 0;
        }
    }
}
};

```

4.6 Tree Diameter

```

const int N = 2e5 + 9;
vector<int> g[N];
int farthest(int s, int n, vector<int> &d) {
    static const int inf = N;
    d.assign(n + 1, inf); d[s] = 0;
    vector<bool> vis(n + 1);
    queue<int> q; q.push(s);
    vis[s] = 1; int last = s;
    while (!q.empty()) {
        int u = q.front(); q.pop();
        for (int v : g[u]) {
            if (vis[v]) continue;
            d[v] = d[u] + 1;
            q.push(v); vis[v] = 1;
        }
        last = u;
    }
    return last;
}
vector<int> dx, dy;
int x = farthest(1, n, dx);
int y = farthest(x, n, dx);

```

5 Math

5.1 FFT

```
const double PI=acos(-1);
struct base{
    double a,b;
    base(double a=0,double b=0):a(a),b(b){}
    base operator+(const base&c) const{return base(a+c.a,b+c.b);}
    base operator-(const base&c) const{return base(a-c.a,b-c.b);}
    base operator*(const base&c) const{return base(a*c.a-b*c.b,a*c.b+b*c.a);}
};
void fft(vector<base>&p,bool inv=0){
    int n=p.size(),i=0;
    for(int j=1;j<n-1;j++){for(int k=n>>1;k>(i^=k);k>>=1);if(j<i)swap(p[i],p[j]);}
    for(int l=1,m;(m=1<<1)<=n;l<<=1){
        double ang=2*PI/m;
        base wn(cos(ang),(inv?1:-1)*sin(ang)),w;
        for(int i=0,j,k;i<n;i+=m)
            for(w=base(1,0),j=i,k=i+1;j<k;j++,w=w*wn){
                base t=w*p[j+1];
                p[j+1]=p[j]-t;
                p[j]=p[j]+t;
            }
    }
    if(inv) for(int i=0;i<n;i++) p[i].a/=n,p[i].b/=n;
}
vector<int> multiply(vector<int>&a,vector<int>&b){
    int n=a.size(),m=b.size(),t=n+m-1,sz=1;
    while(sz<t)sz<<=1;
    vector<base>x(sz),y(sz),z(sz);
    for(int i=0;i<sz;i++){
        x[i]=i<n?base(a[i],0):base(0,0);
        y[i]=i<m?base(b[i],0):base(0,0);
    }
    fft(x);fft(y);
    for(int i=0;i<sz;i++)z[i]=x[i]*y[i];
    fft(z,1);
    vector<int> r(sz);
    for(int i=0;i<sz;i++)r[i]=int(round(z[i].a));
    while(r.size()>1&&r.back()==0)r.pop_back();
    return r;
}
```

```
}
```

5.2 MMGeometricSum

```
vector<int> multi (vector<int> &a, vector<int> &b)
{
    vector<int> ans = {1, 0, 0, 1};
    ans[0] = a[0] * b[0] + a[1] * b[2];
    ans[1] = a[0] * b[1] + a[1] * b[3];
    ans[2] = a[2] * b[0] + a[3] * b[2];
    ans[3] = a[2] * b[1] + a[3] * b[3];
    for (auto &it : ans) it %= mod;
    return ans;
}
vector<int> mat1;
mat1 = {a, 1, 0, 1};
// mat1[0] = a ^ n
// mat1[1] = a^0 + a^1 + ... + a^(n - 1)
void binPowMatrix (int b)
{vector<int> ans = {1, 0, 0, 1};
while (b){if (b & 1){ans = multi(ans, mat1);}
mat1 = multi(mat1, mat1);b >>= 1;}mat1 = ans;}
```

5.3 Mat Xpo

```
struct Mat{
    int n,m;
    vector<vector<int>> a;
    Mat(){}
    Mat(int _n,int _m){n=_n;m=_m;a.assign(n,vector<int>(m,0));}
    Mat(vector<vector<int>> v){n=v.size();m=v[0].size();a=v;}
    inline void make_unit(){assert(n==m);for(int i=0;i<n;i++)
        for(int j=0;j<n;j++)a[i][j]=(i==j);}
    inline Mat operator+(const Mat&b){
        assert(n==b.n&&m==b.m);Mat r(n,m);
        for(int i=0;i<n;i++)for(int j=0;j<m;j++)r.a[i][j]=(a[i][j]
            +b.a[i][j])%mod;
        return r;
    }
    inline Mat operator-(const Mat&b){
        assert(n==b.n&&m==b.m);Mat r(n,m);
        for(int i=0;i<n;i++)for(int j=0;j<m;j++)r.a[i][j]=(a[i][j]
            -b.a[i][j]+mod)%mod;
        return r;
    }
    inline Mat operator*(const Mat&b){
```

```
assert(m==b.n);Mat r(n,b.m);
for(int i=0;i<n;i++)for(int j=0;j<b.m;j++)for(int k=0;k<m;
    k++)
    r.a[i][j]=(r.a[i][j]+1LL*a[i][k]*b.a[k][j]%mod)%mod;
return r;
}
inline Mat fow(int64_t k){
    assert(n==m);Mat r(n,m),b=*this;r.make_unit();
    while(k){if(k&1)r=r*b;b=b*b;k>>=1LL;}
    return r;
}
inline Mat& operator+=(const Mat&b){return *this=(*this)+b;}
inline Mat& operator-=(const Mat&b){return *this=(*this)-b;}
inline Mat& operator*=(const Mat&b){return *this=(*this)*b;}
inline bool operator==(const Mat&b){return a==b.a;}
inline bool operator!=(const Mat&b){return a!=b.a;}
};
```

5.4 NTT

```
const int64_t MOD=998244353,G=3;
inline int64_t modpow(int64_t a,int64_t e){
    int64_t r=1;
    while(e){if(e&1) r=r*a%MOD;
        a=a*a%MOD; e>>=1;
    }
    return r;
}
struct NTT{
    int max_base;
    vector<int64_t> root;
    vector<int> rev;
    NTT(){int tmp=MOD-1; max_base=0;
        while(!(tmp&1)) tmp>>=1,max_base++;
        root.resize(1<<max_base); root[0]=1;
        for(int len=1;len<(1<<max_base);len<<=1){
            int64_t w=modpow(G,(MOD-1)/(len*2));
            for(int i=0;i<len;i++)
                root[len+i]=(i?root[len+i-1]*w%MOD:1);
        }
    }
    void ntt(vector<int64_t>&a,bool inv){
        int n=a.size();
        if((int)rev.size()!=n){
            rev.resize(n); int k=__builtin_ctz(n)-1;
            for(int i=0;i<n;i++) rev[i]=(rev[i>>1]>>1)|((i&1)<<k);
```



```

}
for(int i=0;i<n;i++) if(i<rev[i]) swap(a[i],a[rev[i]]);
for(int len=1;len<n;len<=1)
for(int i=0;i<n;i+=len*2)
for(int j=0;j<len;j++){
int64_t u=a[i+j],v=a[i+j+len]*root[len+j]%MOD;
a[i+j]=(u+v<MOD?u+v:u+v-MOD);
a[i+j+len]=(u-v>=0?u-v:u-v+MOD);
}
if(inv){
reverse(a.begin()+1,a.end());
int64_t inv_n=modpow(n,MOD-2);
for(int64_t&x:a) x=x*inv_n%MOD;
}
}
vector<int64_t> multiply(const vector<int64_t>&a,const
vector<int64_t>&b){
if(a.empty()||b.empty()) return {};
int need=a.size()+b.size()-1,n=1;
while(n<need) n<=1;
vector<int64_t> fa(a.begin(),a.end()),fb(b.begin(),b.end()
);
fa.resize(n); fb.resize(n);
ntt(fa,false); ntt(fb,false);
for(int i=0;i<n;i++) fa[i]=fa[i]*fb[i]%MOD;
ntt(fa,true); fa.resize(need);
return fa;
}
};
static NTT ntt;
vector<int> pow(vector<int>&a,int p){
vector<int>res={1};
while(p){
if(p&1) res=ntt.multiply(res,a);
a=ntt.multiply(a,a); p>>=1;
}
return res;
}
}

```

5.5 Nth Fibonacci

```

map<int64_t, int64_t> dp; // works for n<=1e18
int64_t f(int64_t n) {
if (n <=2) return n >=1; // depends on problem
if (dp.count(n)) return dp[n];
auto a =f(n / 2);
auto b =f(n / 2 + 1);
if (n % 2 ==0) {
dp[n] =(((b*2LL)%mod - (a%mod)+mod))%mod * a)%mod;

```

```

} else {
dp[n] =((a * a)%mod + (b * b)%mod)%mod;
} return dp[n];
}

```

5.6 Rand

```

int rng(){
static std::mt19937 gen(std::chrono::steady_clock::now().
time_since_epoch().count());
return std::uniform_int_distribution<int>(1, INT64_MAX)(
gen);
}

```

5.7 XorBasis

```

const int LOG_K=60;
struct XorBasis{
vector<int64_t> basis;
int64_t N=0,tmp=0;
void add(int64_t x){
N++; tmp|=x;
for(auto&i:basis) x=min(x,x^i);
if(!x) return;
for(auto&i:basis) if((i^x)<i) i^=x;
basis.push_back(x);
sort(basis.begin(),basis.end());
}
int64_t size(){ return basis.size(); }
void clear(){ N=0; tmp=0; basis.clear(); }
bool possible(int64_t x){
for(auto&i:basis) x=min(x,x^i);
return !x;
}
int64_t maxxor(int64_t x=0){
for(auto&i:basis) x=max(x,x^i);
return x;
}
int64_t minxor(int64_t x=0){
for(auto&i:basis) x=min(x,x^i);
return x;
}
int64_t cntxor(int64_t x){
if(!possible(x)) return 0;
return 1LL<<(N-size());
}
int64_t sumOfAll(){

```

```

return tmp*(1LL<<(N-1));
}
int64_t kth(int64_t k){
int64_t sz=size();
if(k>(1LL<<sz)) return -1;
k--; int64_t ans=0;
for(int64_t i=0;i<sz;i++) if(k>>i&1) ans^=basis[i];
return ans;
}
};

```

6 Number Theory

6.1 Factorial Mod

```

//n! % p =?,0(plogp n).
int factmod(int n, int p){
int res = 1;
while(n>1){
res=(res*((n/p)%2? p-1:1))%p;
for(int i=2; i<=n/p; ++i)
res=(res*i)%p;
n/=p; } return res % p; }

```

6.2 Stirling2nd

Stirling number of the second kind is the number of ways to partition a set of n objects into k non-empty subsets and is denoted by $S(n,k)$

$$S(n+1, k) = k * S(n, k) + S(n, k-1)$$

$$S(0, 0) = 1, S(n, n) = 1, S(n, 0) = 0$$

6.3 bigInt

```

struct biginteger {
string s;
biginteger() { s = ""; }
biginteger(string x) { s = x; }
biginteger operator%(const int mod) {
biginteger ret;
int total = 0;
for (int i = 0; i < s.size(); i++) {
total = total * 10 + (s[i] - '0');
total %= mod;
}
}

```

```

    }
    ret.s = to_string(total);
    return ret;
}
biginteger operator+(biginteger t) {
    biginteger ret;
    string ss = s;
    reverse(ss.begin(), ss.end());
    reverse(t.s.begin(), t.s.end());
    while (ss.size() != t.s.size()) {
        if (ss.size() < t.s.size()) ss.push_back('0');
        else t.s.push_back('0');
    }
    int carry = 0;
    for (int i = 0; i < ss.size(); i++) {
        int val = (carry + ss[i] - '0' + t.s[i] - '0');
        ret.s.push_back(val % 10 + '0');
        carry = val / 10;
    }
    if (carry) ret.s.push_back(carry + '0');
    reverse(ret.s.begin(), ret.s.end());
    return ret;
}
biginteger operator*(const biginteger &t) {
    biginteger ret;
    for (int i = 0; i < t.s.size(); i++) {
        if (t.s[i] == '0') continue;
        int dig = t.s[i] - '0';
        int carry = 0;
        biginteger tmp;
        for (int j = s.size() - 1; j >= 0; j--) {
            int val = dig * (s[j] - '0') + carry;
            tmp.s.push_back((val % 10) + '0');
            carry = val / 10;
        }
        if (carry) tmp.s.push_back(carry + '0');
        reverse(tmp.s.begin(), tmp.s.end());
        for (int j = 0; j < t.s.size() - 1 - i; j++) {
            tmp.s.push_back('0');
        }
        ret = ret + tmp;
    }
    return ret;
}
};
biginteger bigmod(biginteger a, int b, int mod) {
    if (b == 1) return a % mod;
    biginteger res = bigmod(a, b / 2, mod);
    res = (res * res) % mod;
    if (b % 2) res = (res * a) % mod;
}

```

```

    return res;
}
void construct(vector<biginteger> &v, vector<biginteger> &
    pref, vector<biginteger> &suf, int n, int mod) {
    for (int i = 0; i < n; i++) {
        v[i] = v[i] % mod;
    }
    pref[0] = v[0] % mod;
    suf[n - 1] = v[n - 1] % mod;
    for (int i = 1; i < n; i++) {
        pref[i] = pref[i - 1] * v[i];
        pref[i] = pref[i] % mod;
    }
    for (int i = n - 2; i >= 0; i--) {
        suf[i] = suf[i + 1] * v[i];
        suf[i] = suf[i] % mod;
    }
}
bool get(int i, vector<biginteger> &v, vector<biginteger> &
    pref, vector<biginteger> &suf, int n, int mod) {
    biginteger val, val2;
    if (i != 0) {
        val = pref[i - 1];
    }
    if (i != n - 1) {
        if (val.s.empty()) {
            val = suf[i + 1];
        } else {
            val = val * suf[i + 1];
            val = val % mod;
        }
    }
    val2 = bigmod(v[i], n - 1, mod);
    return (val.s == val2.s);
}
}

```

6.4 extendEuclid

```

int gcd(int a, int b, int& x, int& y) {if (b == 0) {x = 1; y
    = 0;return a;}}
int x1, y1;int d = gcd(b, a % b, x1, y1);
x = y1; y = x1 - y1 * (a / b);return d;}

```

6.5 phi

```

for (int i = 1; i < N; i++) phi[i] = i;
for (int i = 2; i < N; i++) {

```

```

    if (phi[i] == i) {
        for (int j = i; j < N; j += i) phi[j] -= phi[j] / i;
    }
}

```

6.6 segmented sieve

```

// Generate all primes up to limit using sieve of
    eratosthenes
vector<int> sieve(int limit){
    vector<bool> isp(limit+1,true);
    isp[0]=isp[1]=false;
    for(int p=2;p*p<=limit;p++) if(isp[p])
        for(int i=p*p;i<=limit;i+=p) isp[i]=false;
    vector<int> pr;
    for(int p=2;p<=limit;p++) if(isp[p]) pr.push_back(p);
    return pr;
} // Generate all primes from l to r using segmented sieve
    in O((r - l) log (r) + sqrt(r))
vector<ll> segmented_sieve(ll l,ll r){
    if(l==1) l++;
    int limit=sqrtl(r);
    while(1LL*limit*limit<=r)limit++;
    while(1LL*limit*limit>r)limit--;
    auto pr=sieve(limit);
    vector<bool> isp(r-l+1,true);
    for(ll p:pr){
        ll st=max(p*p,(l+p-1)/p*p);
        for(ll j=st;j<=r;j+=p) isp[j-l]=false;
    }
    vector<ll> v;
    for(ll i=l;i<=r;i++) if(isp[i-l]) v.push_back(i);
    return v;
}

```

7 String

7.1 2D Hashing

```

struct Hashing {
    vector<vector<int>>> hs;
    vector<int> PWX, PWY;
    int n, m;
    static const int PX = 3731, PY = 2999, mod = 998244353;
    Hashing() {}
    Hashing(vector<string>& s) {

```

```

n = (int)s.size(), m = (int)s[0].size();
hs.assign(n + 1, vector<int>(m + 1, 0));
PWX.assign(n + 1, 1);
PWY.assign(m + 1, 1);
for (int i = 0; i < n; i++) PWX[i + 1] = 1LL * PWX[i] *
    PX % mod;
for (int i = 0; i < m; i++) PWY[i + 1] = 1LL * PWY[i] *
    PY % mod;
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        hs[i + 1][j + 1] = s[i][j] - 'a' + 1;
    }
}
for (int i = 0; i <= n; i++) {
    for (int j = 0; j < m; j++) {
        hs[i][j + 1] = (hs[i][j + 1] + 1LL * hs[i][j] * PY %
            mod) % mod;
    }
}
for (int i = 0; i < n; i++) {
    for (int j = 0; j <= m; j++) {
        hs[i + 1][j] = (hs[i + 1][j] + 1LL * hs[i][j] * PX %
            mod) % mod;
    }
}
int get_hash(int x1, int y1, int x2, int y2) { // 1-
    indexed
    assert(1 <= x1 && x1 <= x2 && x2 <= n);
    assert(1 <= y1 && y1 <= y2 && y2 <= m);
    x1--;
    y1--;
    int dx = x2 - x1, dy = y2 - y1;
    return (1LL * (hs[x2][y2] - 1LL * hs[x2][y1] * PWY[dy] %
        mod + mod) % mod -
        1LL * (hs[x1][y2] - 1LL * hs[x1][y1] * PWY[dy] % mod
            + mod) % mod * PWX[dx] % mod + mod) % mod;
}
int get_hash() {
    return get_hash(1, 1, n, m);
}
};

```

7.2 Aho Corasick

```

#define t_sz 26
struct node{
    int ending,link,par,next[t_sz]; vector<int> idx;
    node(){

```

```

        ending=0; par=-1; link=-1; clean(next,-1); } };
struct aho_corasick{
    vector< node > aho;
    aho_corasick(){ aho.emplace_back(); }
    int ID(char ch){ if('a'<=ch && ch<='z') return ch-'a';
        return ch-'A'; }
    void ADD(string &s,int id=0){
        int now,u=0;
        for(auto ch:s){
            now=ID(ch);
            if(aho[u].next[now]==-1) {
                aho[u].next[now]=aho.size(); aho.emplace_back();
                u=aho[u].next[now]; }
            aho[u].ending++; aho[u].idx.push_back(id); }
    int transition(int u, int i) {
        if(u==-1) return 0;
        if(!aho[u].next[i]) return aho[u].next[i];
        return aho[u].next[i]=transition(aho[u].link, i); }
    void push_links(){
        queue<int> q; q.push(0);
        while(!q.empty()){
            int u=q.front(); q.pop();
            for(int i=0; i<t_sz; i++){
                int v=aho[u].next[i]; if(v==-1) continue;
                aho[v].par=u; aho[v].link=transition(aho[u].link,i);
                aho[v].ending+=aho[aho[v].link].ending;
                for(auto &id : aho[aho[v].link].idx)
                    aho[v].idx.push_back(id);
                q.push(v); } } }
    int process(string &s, vector<string> &v){
        //v: list of pattern,int n=v.size();vector<int>pos[n];
        int u=0,sum=0;
        for(int i=0; i<s.size(); i++){
            int x=ID(s[i]); u=transition(u, x);
            sum+=aho[u].ending;
            //for(auto id : aho[u].idx) pos[id].push_back(i);}
            /*for(int i=0; i<n; i++){
                for(auto p : pos[i]){
                    cout<<" "; cout<<p-v[i].size()+1; cout<<endl; }*/
            /*vector< int > st[s.size() + 10]; pref[0] = 1;
            i=0...s.size(): pref[i+1] += pref[i+2-v[id].size()];
            st[i-v[id].size()+1].push_back(v[id].size());
            suff[s.size() + 1] = 1;
            i=s.size()-1 ... 0 //way to make suffix
            for(len : st[i]) suff[i + 1] += suff[i + len + 1];*/
            return sum; }
    void clear(){
        aho.clear();aho.emplace_back(); } };
//min delete from s to avoid the set of string
//pos in s, state in aho-corasick call:solve(0,0)

```

```

int solve(int pos, int state) {
    if(ac.aho[state].ending) return INF;
    if(pos==s.size()) return 0;
    if(!dp[pos][state]) return dp[pos][state];
    int x=solve(pos+1, ac.transition(state, ac.ID(s[pos]));
    int y = 1+solve(pos+1, state);
    return dp[pos][state]=min( x , y); }
main() { aho_corasick ac; ac.ADD(s, i);
    ac.push_links(); ac.process(text); ac.clear();}

```

7.3 Bit Trie

```

const int Log=32;
struct BitTrie{
    int cur; vector<array<int,2>> nxt;
    BitTrie(int n){
        cur=1;
        nxt=vector<array<int,2>>((n+5)*Log,{-1,-1});
    }
    void insert(int x){
        int node=0;
        for(int i=Log-1;i>=0;i--){
            int bt=((x>>i)&1);

            if(nxt[node][bt]==-1) nxt[node][bt]=cur++;

            node=nxt[node][bt];
        }
    }
    int bxor(int x){
        int node=0,res=0;
        for(int i=Log-1;i>=0;i--){
            int bt=((x>>i)&1)^1;

            if(nxt[node][bt]!=-1){
                res|=(1LL<<i);
                node=nxt[node][bt];
            }else if(nxt[node][bt^1]!=-1){
                node=nxt[node][bt^1];
            }else{
                break;
            }
        }
        return res;
    }
};

```

7.4 KMP

```
vector<int> prefix_function(string s){
    int n = (int)s.length(); vector<int> pi(n);
    for(int i = 1; i < n; i++) {
        int j = pi[i-1];
        while(j>0 && s[i] != s[j]) j = pi[j-1];
        if(s[i] == s[j]) j++;
        pi[i] = j; } return pi; }
```

7.5 Longest Palindromes

```
void lp(string& s){
    string t=s;
    reverse(t.begin(),t.end());
    int n=s.size();
    Hash sh(s),th(t);
    auto f=[&](int L, int R){
        auto ss=sh.geth(L,R);
        auto tt=th.geth(n-R+1,n-L+1);
        return (ss==tt);
    };
    vector<int> R(n,1),L(n,1);
    for(int i=n-1,r=n-1;i>=0;i--){
        if(r<n-1 && s[i]==s[r+1]){
            r++;
        }else{
            while(!f(i+1,r+1)){
                r--;
            }
        }
        R[i]=max(R[i],r-i+1);
    }
    for(int i=0,l=0;i<n;i++){
        if(l>0 && s[i]==s[l-1]){
            l--;
        }else{
            while(!f(l+1,i+1)){
                l++;
            }
        }
        L[i]=max(L[i],i-l+1);
    }
}
```

7.6 Min Lexo Rotation

```
int mnlex(string& s){
    int n=s.size(),sz=2*n,l=0,j=1,k=0;
    while(l+k<sz && j+k<sz){
        int a=((l+k)>=n) ? s[l+k-n]: s[l+k];
        int b=((j+k)>=n) ? s[j+k-n]: s[j+k];
        if(a==b){k++;}
        else if(a>b){ // if maxlex then (a<b)
            l=l+k+1; k=0;
            if(l<=j){ l=j+1; }
        }else{
            j=j+k+1; k=0;
            if(j<=l){ j=l+1; }
        }
    }
    return min(l,j);
}
```

7.7 String Hashing

```
const int SZ=int(1e6)+9;
const int B1=151,B2=239;
const int mod1=127657753,mod2=987654319;
vector<int> P1(SZ,1),P2(SZ,1);
void pow_cal(){
    for(int i=1;i<SZ;i++){
        P1[i]=(P1[i-1]*B1)%mod1;
        P2[i]=(P2[i-1]*B2)%mod2;
    }
}
class Hash{
public:
    int n;
    vector<int> H1,H2;
    Hash(const string& s){
        n=s.size();
        H1.assign(n+1,0);
        H2.assign(n+1,0);
        for(int i=1;i<=n;i++){
            H1[i]=(H1[i-1]+((s[i-1]-'a'+1)*P1[i])%mod1)%mod1;
            H2[i]=(H2[i-1]+((s[i-1]-'a'+1)*P2[i])%mod2)%mod2;
        }
    }
    pair<int,int> geth(int l, int r){
        int h1=((H1[r]-H1[l-1]+mod1)%mod1)*P1[SZ-1]%mod1;
        int h2=((H2[r]-H2[l-1]+mod2)%mod2)*P2[SZ-1]%mod2;
        return make_pair(h1,h2);
    }
}
```

```
pair<int,int> geth(){
    return geth(1,n);
}
};
```

7.8 Trie

```
struct TrieNode{
    int32_t nxt[26];
    int32_t freq, pref;
    TrieNode() {
        memset(nxt, -1, sizeof(nxt));
        freq = pref = 0;
    }
};
struct Trie {
    vector<TrieNode> t;
    int64_t totalWords;
    Trie() { clear(); }
    void clear() {
        t.clear();
        t.emplace_back();
        totalWords = 0;
    }
    void insert(const string& s){
        int32_t v = 0;
        t[v].pref++;
        for (char c : s) {
            int32_t x = c - 'a';
            if (t[v].nxt[x] == -1) {
                t[v].nxt[x] = (int32_t)t.size();
                t.emplace_back();
            }
            v = t[v].nxt[x];
            t[v].pref++;
        }
        t[v].freq++;
        totalWords++;
    }
    bool search(const string& s) const{
        int32_t v = 0;
        for (char c : s) {
            int32_t x = c - 'a';
            if (t[v].nxt[x] == -1) return false;
            v = t[v].nxt[x];
        }
        return t[v].freq > 0;
    }
    bool startsWith(const string& s) const{
```

```

int32_t v = 0;
for (char c : s) {
    int32_t x = c - 'a';
    if (t[v].nxt[x] == -1) return false;
    v = t[v].nxt[x];
}
return true;
}
int32_t countPrefix(const string& s) const{
    int32_t v = 0;
    for (char c : s) {
        int32_t x = c - 'a';
        if (t[v].nxt[x] == -1) return 0;
        v = t[v].nxt[x];
    }
    return t[v].pref;
}
void remove(const string& s){
    if (!search(s)) return;
    int32_t v = 0;
    t[v].pref--;
    for (char c : s) {
        int32_t x = c - 'a';
        v = t[v].nxt[x];
        t[v].pref--;
    }
    t[v].freq--;
    totalWords--;
}
int32_t getNode(const string& s) const{
    int32_t v = 0;
    for (char c : s) {
        int32_t x = c - 'a';
        if (t[v].nxt[x] == -1) return -1;
        v = t[v].nxt[x];
    }
    return v;
}
vector<int32_t> nextCharCounts(const string& prefix)
    const{
    vector<int32_t> cnt(26, 0);
    int32_t v = getNode(prefix);
    if (v == -1) return cnt;
    for (int32_t i = 0; i < 26; i++) {
        int32_t u = t[v].nxt[i];
        if (u != -1) cnt[i] = t[u].pref;
    }
    return cnt;
}
int32_t nodeCount() const{ return (int32_t)t.size(); }

```

```

int64_t wordCount() const{ return totalWords; }
};

```

7.9 Z Func

```

vector<int> z_func(string &s){
    int n=s.size();
    vector<int> z(n);
    int l=0,r=0;
    for(int i=1;i<n;i++){
        if(i<r) z[i]=min(r-i,z[i-l]);
        while(i+z[i]<n && s[z[i]]==s[i+z[i]]) z[i]++;
        if(i+z[i]>r){
            l=i;
            r=i+z[i];
        }
    }
    return z;
}

```

7.10 manacharAlgo

```

/*
    ekhane 2*N - 1 length output hoi karon first er ta odd
    length 2nd ta even length
    like aba = a . a . b . a . a
              1 2 1 0 5 0 1 2 1
*/
struct Manacher {
    vector<int> p[2];
    // p[1][i] = (max odd length palindrome centered at i) / 2
    // [floor division]
    // p[0][i] = same for even, it considers the right center
    // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
    Manacher(string s) {
        int n = s.size();
        p[0].resize(n + 1);
        p[1].resize(n);
        for (int z = 0; z < 2; z++) {
            for (int i = 0, l = 0, r = 0; i < n; i++) {
                int t = r - i + !z;
                if (i < r) p[z][i] = min(t, p[z][l + t]);
                int L = i - p[z][i], R = i + p[z][i] - !z;
                while (L >= 1 && R + 1 < n && s[L - 1] == s[R + 1])
                    p[z][i]++, L--, R++;
                if (R > r) l = L, r = R;
            }
        }
    }
}

```

```

}
}
bool is_palindrome(int l, int r) {
    int mid = (l + r + 1) / 2, len = r - l + 1;
    return 2 * p[len % 2][mid] + len % 2 >= len;
}
};
int32_t main() {
    ios_base::sync_with_stdio(0);
    cin.tie(0);
    string s; cin >> s;
    Manacher M(s);
    int n = s.size();
    for (int i = 0; i < n; i++) {
        // 2 multiple hoise karon p emnei create hoise, upure
        // explain kora ache
        cout << 2 * M.p[1][i] + 1 << ' ';
        if (i + 1 < n) cout << 2 * M.p[0][i + 1] << ' ';
    }
    cout << '\n';
    return 0;
}

```

7.11 multiSetHash

```

int n,p1=137,p2=277;
int currPow1[sz+5], currPow2[sz + 5];
int pref1[sz+5], pref2[sz+5];
vector<int> fnArr;
// hash with it's own some space
void pre(){
    currPow1[0]=1;
    currPow2[0]=1;
    for(i,1,sz+4){
        currPow1[i]=(currPow1[i-1]*p1)%mod1;
        currPow2[i]=(currPow2[i-1]*p2)%mod2;
    }
}
void genHash(){
    pref1[0] = 0;
    pref2[0] = 0;
    for(i,0,n){
        // pref[i+1]=(pref[i]*p1+(s[i]-'a'+1))%mod1;
        pref1[i + 1] = (pref1[i] + (fnArr[i] * currPow1[fnArr[i]]) % mod1) % mod1;
        pref2[i + 1] = (pref2[i] + (fnArr[i] * currPow2[fnArr[i]]) % mod2) % mod2;
    }
}
}

```

```

int getHash1(int l,int r){
    return (pref1[r] - pref1[l - 1] + mod1) % mod1;;
}
int getHash2(int l,int r){
    return (pref2[r] - pref2[l - 1] + mod2) % mod2;
}
// hash with some prime pow, unique for all elements
void pre(){
    currPow1[0]=1;
    currPow2[0]=1;
    fr(i,1,sz+4){
        currPow1[i]=(currPow1[i-1]*p1)%mod1;
        currPow2[i]=(currPow2[i-1]*p2)%mod2;
    }
}
void genHash(){
    pref1[0] = 0;
    pref2[0] = 0;
    fr(i,0,n){
        // pref[i+1]=(pref[i]*p1+(s[i]-'a'+1))%mod1;
        pref1[i + 1] = (pref1[i] + (currPow1[fnArr[i]])) %
            mod1) % mod1;
        pref2[i + 1] = (pref2[i] + (currPow2[fnArr[i]])) %
            mod2) % mod2;
    }
}
int getHash1(int l,int r){
    return (pref1[r] - pref1[l - 1] + mod1) % mod1;;
}
int getHash2(int l,int r){
    return (pref2[r] - pref2[l - 1] + mod2) % mod2;
}
// randomize unique for all elements
struct custom_hash {
    static uint64_t splitmix64(uint64_t x) {
        x += 0x9e3779b97f4a7c15;
        x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
        x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
        return x ^ (x >> 31);
    }
    size_t operator()(uint64_t x) const {
        static const uint64_t FIXED_RANDOM = chrono::
            steady_clock::now().time_since_epoch().count();
        return splitmix64(x + FIXED_RANDOM);
    }
}rng;
map<int, int> uniqueVal1, uniqueVal2;
void pre(){
    fr(i,1,sz+4){
        uniqueVal1[i] = rng(i) % mod1;

```

```

        uniqueVal2[i] = rng(i) % mod2;
    }
}
void genHash(){
    pref1[0] = 0;
    pref2[0] = 0;
    fr(i,0,n){
        // pref[i+1]=(pref[i]*p1+(s[i]-'a'+1))%mod1;
        pref1[i + 1] = (pref1[i] + (uniqueVal1[fnArr[i]])) %
            mod1) % mod1;
        pref2[i + 1] = (pref2[i] + (uniqueVal2[fnArr[i]])) %
            mod2) % mod2;
    }
}
int getHash1(int l,int r){
    return (pref1[r] - pref1[l - 1] + mod1) % mod1;;
}
int getHash2(int l,int r){
    return (pref2[r] - pref2[l - 1] + mod2) % mod2;
}

```

7.12 suffixArray

```

const int N=4e5+9,LG=18;
void induced_sort(const vector<int>& vec,int val_range,
    vector<int>& SA,const vector<bool>& sl,const vector<int>
    & lms_idx){
    vector<int> l(val_range,0),r(val_range,0);
    for(int c:vec){if(c+1<val_range)+l[c+1];++r[c];}
    partial_sum(l.begin(),l.end(),l.begin());
    partial_sum(r.begin(),r.end(),r.begin());
    fill(SA.begin(),SA.end(),-1);
    for(int i=lms_idx.size()-1;i>=0;--i) SA[--r[vec[lms_idx[i]
        ]]]]=lms_idx[i];
    for(int i:SA) if(i>=1&&sl[i-1]) SA[l[vec[i-1]]++] = i-1;
    fill(r.begin(),r.end(),0);
    for(int c:vec) ++r[c];
    partial_sum(r.begin(),r.end(),r.begin());
    for(int k=SA.size()-1,i=SA[k];k>=1;--k,i=SA[k]) if(i>=1&&!
        sl[i-1]) SA[--r[vec[i-1]]] = i-1;
}
vector<int> SA_IS(const vector<int>& vec,int val_range){
    int n=vec.size();
    vector<int> SA(n),lms_idx;
    vector<bool> sl(n);
    sl[n-1]=false;
    for(int i=n-2;i>=0;--i){sl[i]=(vec[i]>vec[i+1])||(vec[i]==
        vec[i+1]&&sl[i+1]);if(sl[i]&&!sl[i+1]) lms_idx.
        push_back(i+1);}
}

```

```

reverse(lms_idx.begin(),lms_idx.end());
induced_sort(vec,val_range,SA,sl,lms_idx);
vector<int> new_lms_idx(lms_idx.size(),lms_vec[lms_idx.
    size()]);
for(int i=0,k=0;i<n;++i) if(!sl[SA[i]]&&SA[i]>=1&&sl[SA[i]
    ]-1) new_lms_idx[k++]=SA[i];
int cur=0;SA[n-1]=cur;
for(size_t k=1;k<new_lms_idx.size();++k){
    int i=new_lms_idx[k-1],j=new_lms_idx[k];
    if(vec[i]!=vec[j]){SA[j]=++cur;continue;}
    bool flag=false;
    for(int a=i+1,b=j+1;++a,++b){
        if(vec[a]!=vec[b]){flag=true;break;}
        if((!sl[a]&&sl[a-1])||(!sl[b]&&sl[b-1])){flag=!((!sl[a]
            )&&sl[a-1])&&(!sl[b]&&sl[b-1]));break;}
    } SA[j]=(flag?++cur:cur);
}
for(size_t i=0;i<lms_idx.size();++i) lms_vec[i]=SA[lms_idx
    [i]];
if(cur+1<(int)lms_idx.size()){
    auto lms_SA=SA_IS(lms_vec,cur+1);
    for(size_t i=0;i<lms_idx.size();++i) new_lms_idx[i]=
        lms_idx[lms_SA[i]];
} induced_sort(vec,val_range,SA,sl,new_lms_idx);
return SA;
}
vector<int> suffix_array(const string& s,const int LIM=128){
    vector<int> vec(s.size()+1);
    copy(begin(s),end(s),begin(vec));
    vec.back()='!'; auto ret=SA_IS(vec,LIM);
    ret.erase(ret.begin());
    return ret;
}
struct SuffixArray{
    int n;string s;
    vector<int> sa,rank,lcp; vector<vector<int>>>t; vector<int>
        lg;
    SuffixArray(){}
    SuffixArray(string _s){
        n=_s.size();s=_s;sa=suffix_array(s);
        rank.resize(n);
        for(int i=0;i<n;i++) rank[sa[i]]=i;
        construct_lcp();prec();build();
    }
    void construct_lcp(){
        int k=0;lcp.resize(n-1,0);
        for(int i=0;i<n;i++){
            if(rank[i]==n-1){k=0;continue;}
            int j=sa[rank[i]+1];
            while(i+k<n&&j+k<n&&s[i+k]==s[j+k]) k++;

```

```

    lcp[rank[i]]=k;
    if(k) k--; }
}
void prec(){lg.resize(n,0);for(int i=2;i<n;i++) lg[i]=lg[i/2]+1;}
void build(){
    int sz=n-1;t.resize(sz);
    for(int i=0;i<sz;i++){t[i].resize(LG);t[i][0]=lcp[i];}
    for(int k=1;k<LG;++k) for(int i=0;i+(1<<k)-1<sz;++i) t[i]
        [k]=min(t[i][k-1],t[i+(1<<(k-1))][k-1]);
}
int query(int l,int r){
    int k=lg[r-l+1];
    return min(t[l][k],t[r-(1<<k)+1][k]);
}
int get_lcp(int i,int j){
    if(i==j) return n-i;
    int l=rank[i],r=rank[j];
    if(l>r) swap(l,r);
    return query(l,r-1);
}
int lower_bound(string& t){
    int l=0,r=n-1,k=t.size(),ans=n;
    while(l<=r){
        int mid=l+r>>1;
        if(s.substr(sa[mid],min(n-sa[mid],k))>=t) ans=mid,r=mid-1;
        else l=mid+1; }
    return ans;
}
int upper_bound(string& t){
    int l=0,r=n-1,k=t.size(),ans=n;
    while(l<=r){
        int mid=l+r>>1;
        if(s.substr(sa[mid],min(n-sa[mid],k))>t) ans=mid,r=mid-1;
        else l=mid+1; }
    return ans;
}
pair<int,int> find_occurrence(int p,int len){
    p=rank[p];pair<int,int>ans={p,p};
    int l=0,r=p-1;
    while(l<=r){
        int mid=l+r>>1;
        if(query(mid,p-1)>=len) ans.first=mid,r=mid-1;
        else l=mid+1;}
    l=p+1;r=n-1;
    while(l<=r){
        int mid=l+r>>1;
        if(query(p,mid-1)>=len) ans.second=mid,l=mid+1;

```

```

        else r=mid-1; }
    return ans;
}
};
void solve(){
    int n,m,q;cin>>n>>m>>q;
    string a,b;cin>>a>>b;
    string s=b+'@'+a+'#';
    SuffixArray sfa(s);
    vector<int>maxSub(m+1,0);
    for(int i=1;i<s.size();i++){
        if(sfa.sa[i]<m){
            int nw1=sfa.lcp[i-1],nw2=inf;
            if(sfa.sa[i-1]<m) nw2=maxSub[sfa.sa[i-1]+1];
            maxSub[sfa.sa[i]+1]=min(nw1,nw2);} }
    for(int i=s.size()-2;i>=0;i--){
        if(sfa.sa[i]<m){
            int nw1=sfa.lcp[i],nw2=inf;
            if(sfa.sa[i+1]<m) nw2=maxSub[sfa.sa[i+1]+1];
            maxSub[sfa.sa[i]+1]=max(maxSub[sfa.sa[i]+1],min(nw1,nw2)); }
    }
} build(m,maxSub);
while(q--){
    int l,r;cin>>l>>r;
    int lo=1,hi=r-l+1,mid,ans=0;
    while(lo<=hi){
        mid=(lo+hi)/2;
        int nw=query(l,r-mid+1);
        if(nw>=mid) ans=mid,lo=mid+1;
        else hi=mid-1; }
    cout<<ans<<endl; } }
// ekhane lcp 1 len kom thake and lcp[i] means at sorted
// array, at index i + 1, it has a common prefix of len
// lcp[i]

```

7.13 suffix_array2

```

void suffix_array(string s){
    s+="\$";
    int32_t n=s.size(),N=max(n,int32_t(260));
    vector<int32_t> sa(n),R(n);
    for(int32_t i=0;i<n;i++) sa[i]=i, R[i]=s[i];
    for(int32_t k=0;k<n;k?(k<=1):k++){
        vector<int32_t> nR(n),cnt(N);
        auto nsa=sa;
        for(int32_t i=0;i<n;i++) nsa[i]=(nsa[i]-k+n)%n, cnt[R[i]]++;
        for(int32_t i=1;i<N;i++) cnt[i]+=cnt[i-1];
    }
}

```

```

    for(int32_t i=n-1;i>=0;i--) sa[--cnt[R[nsa[i]]]]=nsa[i];
    for(int32_t i=1,r=0;i<n;i++){
        r+=(R[sa[i]]!=R[sa[i-1]] || R[(sa[i]+k)%n]!=R[(sa[i-1]+k)%n]);
        nR[sa[i]]=r;
    }
    swap(R,nR);
}
vector<int32_t> lcp(n-1);
for(int32_t i=0,k=0;i<n-1;i++){
    int32_t pi=R[i];
    int32_t j=sa[pi-1];
    while(i+k<n && j+k<n && s[i+k]==s[j+k]) k++;
    lcp[pi-1]=k;
    if(k) k--;
}
}

```

8 ZMiscellenous

8.1 build

```

{
    "cmd" : ["g++ -std=c++20 $file_name -o $file_base_name &&
        timeout 5s ./ $file_base_name<input.txt>output.txt"],
    "selector" : "source.cpp",
    "shell": true,
}

//orderSet
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
#define ordered_set tree<int, null_type,less<int>,
    rb_tree_tag,tree_order_statistics_node_update>

//STDIN
freopen("mj5.in", "r", stdin); freopen("mj5.out", "w",
    stdout);

```

8.2 formulas

Sum of n natural numbers = $n * (n + 1) / 2$;
 Sum of n natural squared numbers = $n * (n + 1) * (2 * n + 1) / 6$;


```

Sum of n natural cube numbers = (n * (n + 1) / 2)^2;
Sum of first n even numbers = n * (n + 1)
Sum of first n odd numbers = n ^ 2;
Chord Length = 2 * ( r - d )
area of a sector/arc = (/360) * r ( is in degree)
area of a sector/arc = (1/2) * * r ( is in radians)
sine rule a/sin A = b/sin B = c/sin C
cosine rule c = a + b - 2ab cos C
sum of angles of a polygon (n - 2) * 180
Catalen numbers Cn = (1 / (n + 1)) * (2n n) = (2n n) - (2n (
    n + 1)) = sumOf i to n (Ci * C(n-i))
Area of isoceles = .5 * a * b * sin(c)
Area of any triangle = sqrt(s(s-a)(s-b)(s-c))
Total angle of polygon = (n-2)*180
Arithmetic series nth number, an = a1 + (n - 1) * d

```

```

Arithmetic series sum, S_n = (n/2) * (a_1 + a_n)
Geometric series nth number, an = a1 * r^(n-1)
Geometric series sum, S_n = a* (1 - r^n) / (r - 1)
Area of trapezium without height = 1/2 * (a + b) * h
Right circular cone volume, V = PI * r^2 * (h / 3)
conical frustum, V = (/3) * height * (r1 * r1 + r2 * r2 + (
    r1 * r2))
* (a | b) =(a^b)|(a&b)      * a^(a&b) =(a|b)^b
* b^(a&b) =(a|b)^a      * (a&b)^(a|b) =(a^b)
* (a+b) =(a^b) + 2*(a&b)  * (a+b) =(a|b) + (a&b) //plus
* (a-b) =(a^(a&b))-((a|b)^a) * (a-b) =(a^(a&b))-(b^(a&b))
* (a-b) =((a|b)^b)-(b^(a&b)) * (a-b) =((a|b)^b)-((a|b)^a)
1e18+21, 1e18+33, 1e18+87, 1e18+93, 2e18+11, 2e18+33,
2e18+63, 2e18+87

```

```

const int Primes[50] = {2487161969, 3856247663, 3158177263,
4443359849, 2928712063, 1534911193, 3704798833,
3858635329, 1406144053, 1799020313, 4161289499,
4808093141, 1074412681, 1622517073, 1910710603,
3563848823, 4965109181, 3028205659, 1332566561,
1443494453, 3972264077, 2796138953, 1811808701,
3995829011, 3264371483, 3284732653, 2252592907,
2944218709, 1195921303, 1240390289, 3338920877,
2016761807, 2529498863, 952181821, 4333778233,
4693567763, 4688538703, 4905109823, 4043754641,
2731077191, 3889851109, 2314277087, 4110023501,
2548368847, 3368441227, 1232754433, 4611307861,
1024710161, 3398593411, 3489896321};

```